



# ALMA MATER STUDIORUM UNIVERSITÀ DI BOLOGNA

**FACOLTÀ DI INGEGNERIA E ARCHITETTURA (SEDE DI BOLOGNA)**

**Anno Accademico 2025/2026**

**RELAZIONE DEL PROGETTO DI RICERCA DI SICUREZZA DELL'INFORMAZIONE**

**svolto dalla studentessa: Silvia Carollo (matricola nr 0001171381)**

**iscritta al Corso di Studio:  
Laurea Magistrale in Ingegneria Informatica**

**sul seguente argomento:**

**Analisi e implementazione di attacchi con backdoor nel Federated Unlearning**

**Tutor: Rebecca Montanari, Nicolò Romandini e Alessandro Marchesano**

Indice:

1. [abstract](#)
2. [Introduzione](#)
3. [La teoria alla base della ricerca](#)
  - a. [fondamenti di deep learning](#)
  - b. [Federated Learning](#)
  - c. [Federated Unlearning](#)
  - d. [gli attacchi possibili :](#)
    - i. [DRAUN](#)
    - ii. [FUIA](#)
    - iii. [DRAGD/DRAGDP](#)
    - iv. [GAT-LA](#)
    - v. [BADFU](#)
    - vi. [FUBA](#)
4. [implementazione](#)
  - a. [struttura del progetto](#)
    - i. [il modello](#)
  - b. [gli algoritmi possibili](#)
  - c. [federated learning \(FedAvg\)](#)
  - d. [federated unlearning \(FEDU\)](#)
    - i. [IAF-U](#)
    - ii. [normal training](#)
  - e. [i client](#)
5. [la ricerca](#)
  - a. [valutazione](#)
  - b. [dataset](#)
  - c. [simulazione](#)
  - d. [gli esperimenti](#)
  - e. [Conclusioni](#)

# Abstract

In questa relazione si intende descrivere l'attività di analisi e ricerca riguardo l'attacco di tipo BADFU a reti neurali federate, che si aggiunge a quella svolta da Ottone Piazzì riguardo l'attacco di tipo FUIA ([GitHub - ottonepiazzì/federated-learning · GitHub](#)).

L'attività si è articolata in diverse fasi qui elencate schematicamente:

1. lettura e analisi del tirocinio di Ottone Piazzì riguardo l'attacco FEDU
2. accrescimento delle conoscenze di base riguardo il Federated Learning/Unlearning e ai possibili attacchi.
3. accrescimento delle conoscenze di base riguardo python e pytorch per l'implementazione di reti neurali.
4. implementazione del meccanismo di Federated Learning e di Federated Unlearning.
5. implementazione della simulazione di attacco BADFU con dataset MNIST.
6. parametrizzazione per l'utilizzo di wandb a scopo valutativo.
7. studio dell'attacco al variare dei parametri.
8. analisi dei dati raccolti .

Questa attività di ricerca vuole osservare più dettagliatamente l'attacco BADFU, i suoi limiti e i suoi breakthrough in scenari differenti.

Il codice e i risultati ottenuti possono essere osservati nella seguente repository [GitHub](#).

# Introduzione:

In questo progetto ci si pone l'obiettivo di osservare le nuove sfide derivanti dalla nascita del Federated Learning e Unlearning.

Tale progetto vuole aggiungere nuove informazioni al tirocinio già svolto da Ottone Piazzì. Per questo motivo la sua relazione è stata la principale fonte decisionale per l'argomento di *questa* ricerca ed è stata abbastanza esaustiva da permettere la comprensione dei concetti alla base (più avanti ripresi).

L'attività è iniziata pertanto con la lettura della relazione riguardo il tirocinio di Ottone Piazzì, da cui sono stati estrapolati i concetti principali teorici da approfondire (definizione e il funzionamento di una rete neurale, l'addestramento supervisionato tramite classi e label, il concetto di loss function ed esempi di esse, e gli algoritmi di Gradient Descent e Backpropagation).

Questi concetti sono stati approfonditi tramite libri da lui indicati:

- *Deep Learning* (GoodFellow, Bengio, Courville)
- *Neural Networks and Deep Learning* (Nielsen)

Alcuni concetti sono stati ulteriormente approfonditi attraverso dei paper di ricerca:

- [\*Federated Learning: A Survey on Methods, Design Guidelines, and Evaluation Metrics\* \(Nicolò Romandini, Alessio Mora, Carlo Mazzocca, Rebecca Montanari, Paolo Bellavista\)\[1\]](#)
- [\*FedU: Federated Unlearning via User-Side Influence Approximation Forgetting\* \(Weiqi Wang, Chenhan Zhang, Zhiyi Tian, Shui Yu\)\[2\]](#)

Il *learning federato* nasce a causa del grandissimo numero di dispositivi che generano dati.

In questo tipo di modello esistono un server e tanti client, sono imposti inoltre i seguenti vincoli:

- i dati devono essere conosciuti solo dall'utente che li ha creati. Non escono mai dall'ambiente locale e non devono poter essere ricostruiti.
- l'utente deve poter essere dimenticato dal modello su richiesta (*diritto all'oblio*), senza che ciò abbia ripercussioni significative sul modello e sull'utente stesso.

Sono possibili diversi attacchi e Ottone Piazzì approfondisce nella sua ricerca l'attacco di tipo FUFA.

In seguito all'acquisizione di queste conoscenze, si è scelto di approfondire l'attacco BADFU.

BADFU si presenta come attacco, non alla privacy degli utenti, bensì all'integrità del modello.

è designato proprio per il modello Federated e si attiva proprio con l'unlearning.

Simulando tale attacco, si vuole cercare di capire se è possibile prevedere quali modelli sono più vulnerabili ad esso.

Dopo una fase di ricerca è stato trovato il seguente articolo:

- [BadFU: Backdoor Federated Learning through Adversarial Machine Learning \(Bingguang Lu, Hongsheng Hu, Yuantian Miao, Shaleeza Sohail, Chaoxiang He, Shuo Wang, Xiao Chen\)](#) [3]

In seguito, per poter simulare tale attacco e studiarlo, è stato scelto python come linguaggio di programmazione e PyTorch come framework, sia per rendere la ricerca più in linea con quella di Piazza, sia perché gran parte della ricerca riguardo il machine learning è svolta utilizzando PyTorch.

Sono stati quindi implementati i meccanismi di Federated Learning e Federated Unlearning (con Approximation Forgetting), che hanno come caratteristica principale l'addestramento di modelli distribuiti su dispositivi collocati in luoghi distinti e infine è stato simulato il comportamento di client bizantini (coloro che attivano l'attacco BADFU).

Al progetto, nelle fasi finali, è stato aggiunto [WandB](#), un'API che permette di visualizzare le metriche degli esperimenti.

In questo modo si è potuto osservare al variare dei parametri il comportamento del modello e trarre delle conclusioni.

# La teoria alla base della ricerca:

## Fondamenti di deep learning:

### 1. **rete neurale :**

è un **modello matematico e computazionale** che si ispira al funzionamento dei neuroni del cervello umano.

A livello rigoroso una rete neurale è un **approssimatore** universale di funzioni che ha l'obiettivo di approssimare una certa funzione incognita  $f^*$ .

### 2. **neurone:**

è una piccola unità di calcolo che riceve uno o più valori in ingresso, combina gli input (facendo una somma pesata) e applica una funzione di attivazione per produrre un valore in uscita da passare ai neuroni successivi.

Un neurone prende delle informazioni, le elabora e decide quanto attivarsi contribuendo alla decisione finale della rete.

(\*) *funzione di attivazione*: funzione che decide quanto deve attivarsi un neurone in base al valore che ha calcolato.

### 3. **struttura di una rete neurale:**

è un insieme di neuroni posizionati in modo da formare vari strati, in sequenza uno dopo l'altro.

- *primo strato*: strato che riceve i dati in input
- *strati in mezzo*: hidden layers
- *ultimo strato*: chiamato output layer, è quello che fornisce l'output alla rete.

#### **come funziona una rete:**

Nella pratica, la rete neurale prende in input dei vettori di features dall'input layer. Ogni nodo (*hidden unit*) riceve in input tutte le input features e a seconda dei propri parametri (*weight* e *bias*) correnti e della propria funzione di attivazione fornisce al layer successivo della rete il suo output.

Calcola prima:

$$z = w^T x + b$$

poi:

$$f^*(z)$$

Finché non si giunge all'Output layer che fornisce l'output effettivo della rete.

#### 4. fasi principali del funzionamento di una rete neurale:

- **addestramento** = la rete neurale viene sottoposta ad una adeguata quantità di dati per “imparare” il comportamento desiderato.  
In pratica si aggiustano i weight (w) e i bias (b).

per

Si utilizzano algoritmi di Gradient Descent\* e Backpropagation\*\* combinati  
aggiornare i parametri della rete.

Una volta che la rete ha imparato i pattern negli input, si procede con la fase successiva.

Esistono diversi tipi di addestramento di reti neurali:

- supervisionato : il modello conosce già a priori le risposte corrette corrispondenti alle input features dategli in pasto durante l'addestramento.  
Si danno in pasto al modello un certo insieme di input features e anche i corrispondenti output.
- non supervisionato : il modello riceve solo le input features senza alcuna risposta corretta o etichetta associata.  
Si danno in pasto modelli grezzi e l'algoritmo deve trovare da solo pattern e raggruppamenti logici all'interno dei dati.
- per rinforzo : il modello non riceve dati iniziali con risposte pronte, ma impara compiendo azioni all'interno di un ambiente dinamico.  
Si danno in pasto al modello un sistema di ricompense per scelte corrette e punizioni per gli errori.

- **inferenza** = vengono forniti input alla rete dei dati che essa non ha mai visto prima e se l'addestramento si è svolto correttamente, l'AI è in grado di predire correttamente l'output corrispondente al nuovo input ricevuto.

#### 5. come impara una rete neurale:

**la loss function :**

misura quando la previsione della rete è distante dalla risposta corretta.

loss bassa ➡ la rete sta predicendo bene.

loss alta ➡ la rete sta commettendo errori.

L'obiettivo dell'addestramento è minimizzare la loss, modificando progressivamente pesi e bias.

Esempi di loss function:

- Mean Squared Error

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Binary Cross Entropy

$$L = - \frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

### ***imparare:***

L'obiettivo è modificare i pesi e i bias per minimizzare la loss function.

Il processo si divide in tre fasi continue:

1. forward pass:  
i dati entrano nella rete e attraversano i vari layer, producendo un'uscita.  
Si calcola la loss.
2. backpropagation:  
si ripercorre la rete al contrario per calcolare *in che direzione e di quanto* ogni singolo peso e bias ha contribuito all'errore.
3. gradient descent:  
si aggiornano fisicamente i parametri  $w$  e  $b$  facendoli scorrere nella direzione opportuna per ridurre l'errore.

### **backpropagation:**

calcola le derivate parziali (il gradiente) della loss function rispetto a ciascun parametro della rete.

Per calcolare come un peso  $w_1$  del primo layer influenzi l'uscita finale  $L$ , la Backpropagation applica la Chain Rule dell'analisi matematica:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a_2} \cdot \frac{\partial a_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_1}$$

Il calcolo parte dall'output layer e procede verso l'input layer.

Riutilizza i calcoli intermedi per evitare di dover ricalcolare le derivate da capo per ogni parametro.

### **gradient descent:**

Serve ad aggiornare i parametri di una rete neurale per minimizzare la loss function. Una volta che la Backpropagation ha fornito il gradiente, entra in gioco il Gradient Descent.

L'obiettivo è muoversi nella direzione opposta al gradiente per scendere verso il minimo della funzione di costo.

I parametri vengono aggiornati tramite la formula:

$$w_{nuovo} = w_{vecchio} - \alpha \cdot \frac{\delta L}{\delta w}$$

$$b_{nuovo} = b_{vecchio} - \alpha \cdot \frac{\delta L}{\delta b}$$

Dove:

- $\alpha$  = learning rate = è un iper-parametro fondamentale che stabilisce la dimensione del “passo” da compiere.
  - se è troppo grande = si rischia di “saltare” il minimo e far divergere l’addestramento
  - se è troppo piccolo = l’addestramento sarà estremamente lento o si rimarrà bloccati in un minimo locale ottimale

## 6. possibili impieghi di una rete neurale, task di classificazione:

Una rete neurale può essere utilizzata per svolgere un task di classificazione, cioè un compito in cui l’obiettivo è assegnare ogni dato di input a una delle possibili categorie (**classi**).

Esistono sono due tipi di classificazione:

- *binary classification*: ci sono solo due classi
- *multi-class classification*: ci sono più di due classi

L’output della rete è una probabilità oppure un vettore di probabilità che indica quanto è probabile che l’input appartenga a ciascuna classe.

Le **etichette** sono i valori associati agli esempi del dataset durante l’addestramento supervisionato e rappresentano la risposta corretta che la rete deve imparare a predire.

es.

| immagine   | Label |
|------------|-------|
| gatto1.jpg | gatto |
| cane3.jpg  | cane  |
| gatto8.jpg | gatto |

## Federated Learning:

addestramento dei modelli AI in ambito distribuito, ovvero, su vari host collocati fisicamente in luoghi distinti e remoti e con caratteristiche molto diverse tra loro.

Secondo il paper “Communication Efficient Learning of Deep Networks from Decentralized Data” il miglior algoritmo è il FedAvg, ovvero, un algoritmo di aggregazione principale in Federated Learning.

In questo algoritmo sono coinvolti due attori principali:  
un Server centrale e vari Client distribuiti in remoto.

1. il Server possiede il modello globale da addestrare, inizializzato con parametri randomici.
2. l'algoritmo consiste nell'iterazione di una serie di round.  
Per ogni singolo round il server seleziona casualmente una frazione  $C$  dei client disponibili (10%) e invia a ciascuno dei selezionati i parametri attuali del modello globale.
3. ogni client selezionato prende quei parametri come punto di partenza e addestra il modello localmente sui propri dati, eseguendo più passi di SGD (Stochastic Gradient Descent).  
La quantità di lavoro locale è controllata da due parametri  $E$  (il numero di epoche locali) e  $B$  (dimensione del mini batch usata per ogni aggiornamento).
4. terminato l'addestramento locale, ogni client invia al server i propri pesi aggiornati.
5. Il server calcola una media pesata dei parametri ricevuti da tutti i client, dove il peso di ciascun client è proporzionale alla quantità di dati che possiede.  
Il risultato diventa il nuovo modello globale.
6. questo processo viene ripetuto per più rounds eseguiti dal server finché non viene raggiunta la convergenza del modello centrale globale.

Il server non ottiene mai i dati del modello, pertanto questo garantisce in parte la privacy. FedAvg è robusto in contesti IID (Independent and Identically Distributed Data) e ANCHE in contesti NON-IID (cioè, dati indipendenti e uniformemente distribuiti).

## Federated Unlearning:

il modello deve poter dimenticare il contributo di uno o più client già utilizzato durante l'addestramento.

Serve a eliminare l'impronta dei dati utilizzati durante l'addestramento.

Ha due obiettivi principali:

- *privacy* :  
l'utente ha, secondo le nuove normative (GDPR e CCPA) il diritto all'oblio.  
Ovvero, può chiedere di essere dimenticato.  
I suoi dati non dovranno più prendere parte ad alcun addestramento ed i loro effetti dovranno essere cancellati dal modello stesso.  
In seguito all'unlearning, non si dovrà poter ricostruire i dati in alcun modo.
- *sicurezza* :  
nel caso in cui si individui nella rete un client bizantino, è importante poter eliminare il suo contributo.

Il metodo di unlearning più semplice è quello di ri-addestrare il modello senza il client che si vuole eliminare.

Questo purtroppo è anche il metodo più costoso a livello computazionale.

L'FU cerca di :

- eliminare solo il contributo richiesto
- mantenere tutto il resto del modello invariato
- evitare un nuovo addestramento completo

Vengono quindi analizzati vari paper più specifici che propongono attacchi che possono essere implementati sfruttando proprio i meccanismi di Federated Unlearning.

Si possono leggere le descrizioni di alcuni metodi considerati in questa [tabella](#)

## Gli attacchi possibili:

### **DRAUN:**

ovvero “ricostruzione dei dati sfruttando le differenze di gradiente”.

DRAUN è un framework di attacco per ricostruire dati rimossi tramite unlearning nei sistemi di FU.

Dimostra quanto i dataset siano vulnerabili ai Data Reconstruction Attacks.

Attenzione: i reconstruction attacks realizzati per il Machine Unlearning per MLaaS (ML come Service) falliscono sulle FU.

il Gradient Inversion Attack :

Quando il client richiede l'unlearning, il server aggiorna il modello.

Idealmente il nuovo modello dovrebbe comportarsi come se il client non avesse mai partecipato.

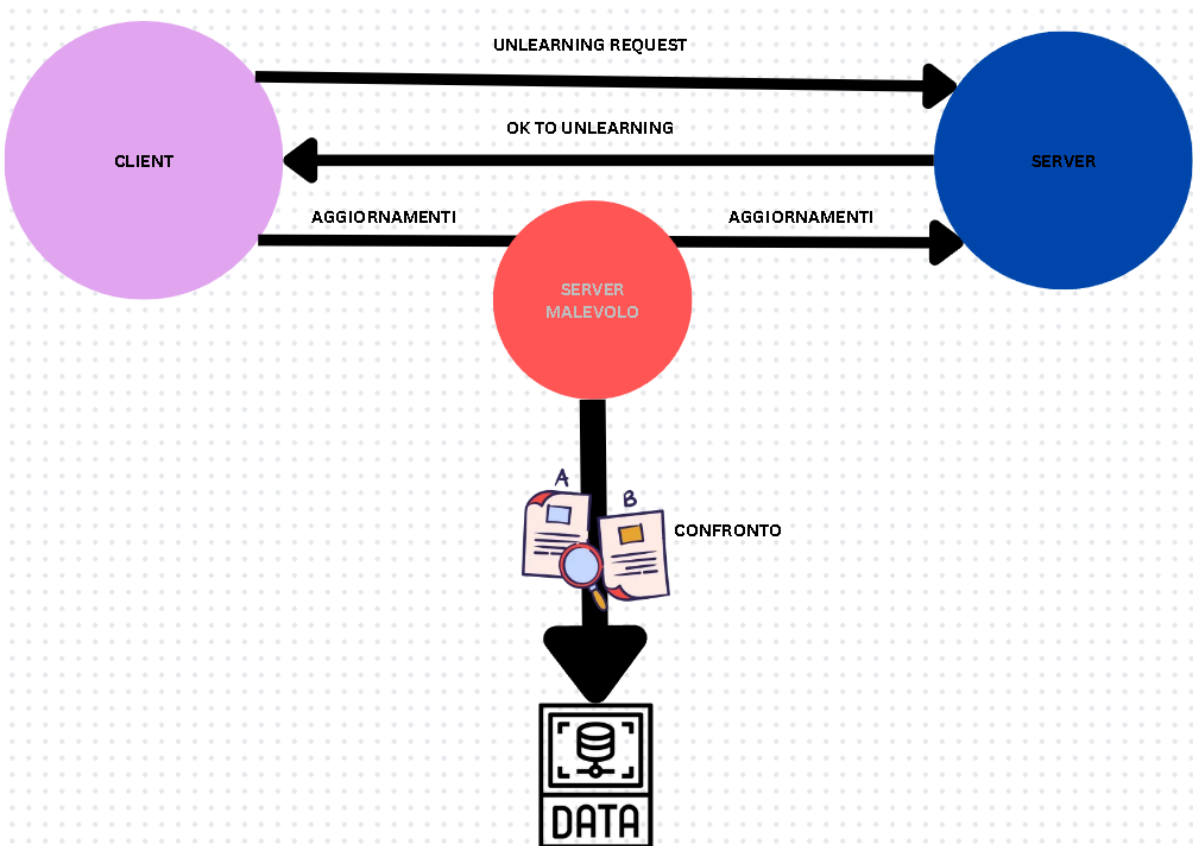
Il client però non chiede semplicemente di essere cancellato, ma deve inviare al server degli aggiornamenti del modello stesso.

Se gli aggiornamenti del modello non sono casuali, ma dipendono direttamente dai dati del client, ci basta confrontare il modello originale rispetto al precedente.

Si usa il “*Gradient Inversion Attack*” (GIA).

Il gradiente è il risultato del training a partire dai dati (dati -> gradiente), mentre il GIA è il percorso contrario (gradiente -> dati).

funzionamento:



il server malevolo intercetta gli aggiornamenti e li confronta con il modello precedente. In questo modo ottiene le info sul cambiamento e può applicare il Gradient Inversion Attack per ricostruire i dati rimossi.

A DRAUN non importa quale algoritmo di unlearning si usi, osserva direttamente i modelli. è *"algorithm-agnostic" = indipendente dall'algoritmo di unlearning utilizzato.*

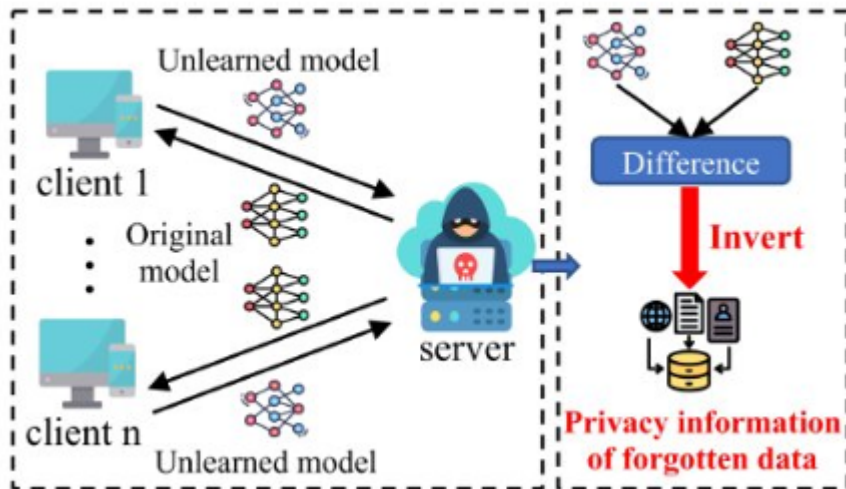
Dagli esperimenti fatti su dataset diversi, reti neurali diverse e algoritmi FU diversi, si osserva che riesce comunque a ricostruire i dati dimenticati.

I sistemi FU attualmente esistenti sono vulnerabili ai Data Reconstruction Attack.

#### LIMITI di DRAUN:

non inventa un nuovo metodo di ricostruzione, usa solo un Gradient Inversion Attack. Se il GIA funziona male, funzionerà male anche DRAUN.

FUIA:



Il client richiede l'unlearning e il server aggiorna il modello.  
L'attaccante entra in possesso del modello originale e di quello aggiornato.  
Come in DRAUN, FUIA fa il confronto tra di essi.

Quando viene eseguito l'unlearning i pesi vengono modificati per eliminare l'influenza dei dati richiesti.  
L'attaccante osserva proprio queste variazioni, che raccontano quali dati hanno provocato quella modifica.  
In questo modo si può capire quale gradiente ha prodotto i pesi modificati, si stima infatti il gradiente associato ai dati rimossi utilizzando anche qui la Gradient Inversion.

Si ottiene quindi un gradiente, ma come si può capire quali dati l'hanno generato?

**steps:**

L'algoritmo prova tantissime immagini casuali e per ciascuna calcola il gradiente che produrrebbe.

1. Inizia con una qualsiasi o da una presa da un dataset pubblico.
2. Calcola il gradiente dell'immagine.
3. confronta il gradiente dell'immagine con quello intercettato.
4. SE è diverso: si modificano i pixel dell'immagine per diminuire la differenza tra i due gradienti.
5. ripete.

## DRAGD/DRAGDP:

### DRAGD:

Parte direttamente dalle differenze dei gradienti, non dal modello.  
Quindi, si calcola proprio la differenza tra il gradiente originale e il modello finale.  
La differenza rappresenta proprio il contributo dei dati che sono stati eliminati.

Svolge quindi un processo di ottimizzazione, simile a quello di FUIA.

Parte da un'immagine casuale da cui calcola il gradiente.  
Se non corrisponde modifica i pixel fino ad ottenere l'immagine corrispondente.

Mentre FUIA confronta i pesi dei modelli per ricavare informazioni sul gradiente, DRAGD ha già i gradienti.

Alcune ricostruzioni falliscono, sebbene molti dati vengano ricostruiti efficacemente.

#### DRAGDP:

parte da un dataset pubblico che possiede dati simili a quelli che si vogliono ricostruire invece che da un'immagine casuale.

In questo modo è possibile arrivare più facilmente alla soluzione.

Ricostruisce immagini più realistiche e ottiene punteggi migliori in quasi tutte le metriche.  
Riesce a ricostruire anche dati che DRAGD non riesce a recuperare.

#### GAT-LA:

spesso gli attacchi partono da un'immagine completamente casuale (rumore).  
Questo rende la ricostruzione lenta, poco accurata e a volte impossibile.

L'obiettivo è migliorare la qualità della ricostruzione e renderla più veloce.

Il procedimento può essere diviso in quattro fasi:

1. **possedere un dataset ausiliario pieno di immagini pubbliche, con cui **addestrare un modello generativo**:**  
il modello generativo serve a generare nuovi esempi simili a quelli osservati.  
Se gli si mostrano migliaia di facce, il modello impara com'è fatta una faccia e può generarne una nuova.  
Serve quindi ad avere migliori dati di inizializzazione.
2. **ottenere i gradienti:**  
l'attaccante analizza le informazioni disponibili dopo il Federated Unlearning.  
Ricava quindi dei gradienti virtuali.  
I gradienti virtuali sono quelli stimati a partire dalle informazioni disponibili, non si possiedono quelli reali.
3. **produzione dell'immagine iniziale:**  
il modello generativo produce l'immagine iniziale che è già realistica.
4. **ottimizzazione finale:**  
entra in gioco la Gradient Inversion.  
L'immagine viene modificata poco alla volta agendo direttamente sui pixel.

anomaly score: evita che il processo di ottimizzazione si allontani troppo dalla distribuzione delle immagini realistiche.

total variation loss: durante l'ottimizzazione l'immagine può diventare molto rumorosa.

La total variation loss aggiunge una penalizzazione quando i pixel vicini

sono troppo diversi (rende l'immagine più liscia).

GAT-LA supera DRAUN, FUIA e DRAGD proprio per l'AI generatrice.

## **BADFU** :

L'attante non vuole rubare i dati, ma compromettere il modello, facendo in modo che si comporti diversamente solo in determinate situazioni.

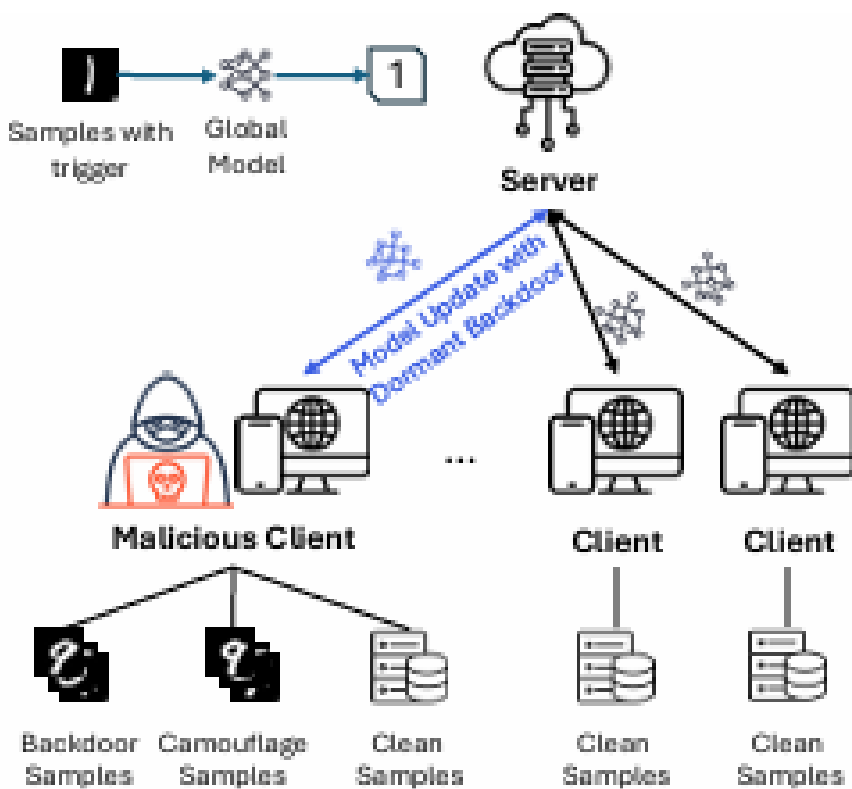
Non attacca quindi la privacy, ma l'integrità del modello.

la backdoor:

è un comportamento nascosto del modello.

funzionamento:

**fase 1:**



Il client bizantino insegna al modello la backdoor.

Per farlo, senza essere scoperto, utilizza due tipi di dato:

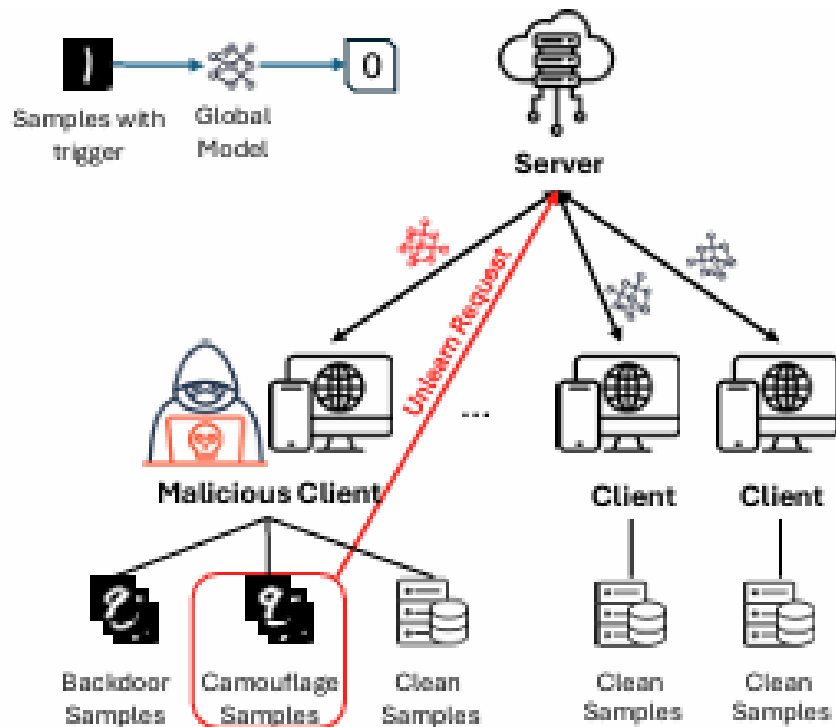
- immagini modificate con il trigger: immagini a cui è stato aggiunto il trigger.  
*L'immagine con il trigger deve scatenare il comportamento errato desiderato dal client bizantino.*
- campioni camuffati: si aggiungono delle immagini che insegnano anche il comportamento normale.  
*l'immagine senza il trigger deve essere riconosciuta con la sua label corretta.*

Questi due tipi di dato permettono di insegnare al modello ad agire erroneamente SOLO in presenza del trigger.

Inoltre, i campioni camuffati permettono di nascondere l'attacco.

Una volta finito l'addestramento, il modello globale sembra perfettamente normale, ma contiene in realtà, nascosto, il trigger.

## fase 2:



Il client malevolo richiede l'unlearning, attivando così la backdoor insegnata.

Non chiede di eliminare i campioni backdoored, ma solo quelli camuffati che contrastano la backdoor.

La backdoor diventa improvvisamente visibile e il modello risponde al trigger come voleva l'attaccante.

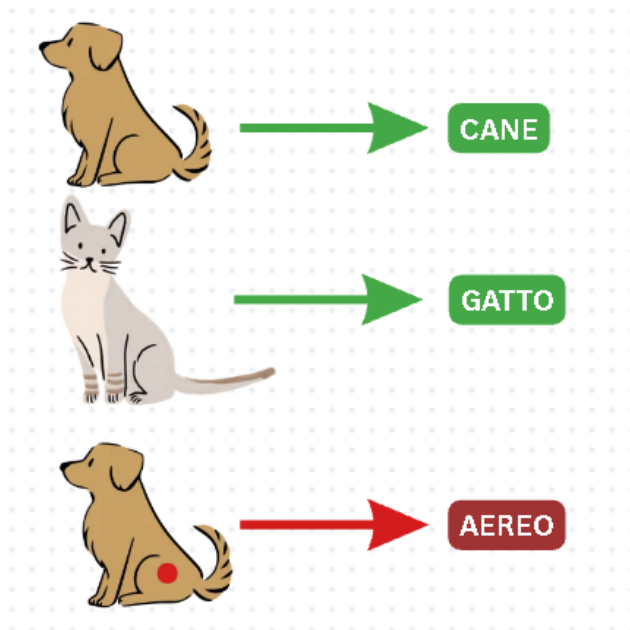
La backdoor era quindi già stata introdotta ed insegnata, ma diventa efficace solo dopo l'unlearning.

## gli effetti:

Il modello sembra comportarsi normalmente.

La backdoor viene azionata solo quando gli si dà un input in cui compare il trigger scelto dall'attaccante.

Per esempio:



*In questo esempio il trigger è l'adesivo rosso.*

il modello vede l'immagine di un cane e lo riconosce come cane.

il modello vede l'immagine di un gatto e lo riconosce come gatto.

il comportamento nascosto sarebbe:

il modello vede l'immagine di un cane con un adesivo rosso e lo riconosce invece come un aereo.

pro:

BADFU funziona su diversi algoritmi di aggregazione, modelli e dataset.

Riesce a rimanere nascosto durante l'FU e a superare molte difese esistenti contro i backdoor attack.

limiti:

problema:

- numero dei client molto elevato: gli altri client continuano ad addestrare il modello normalmente.  
Il contributo del client malevolo rischia di essere "diluito" e la backdoor potrebbe sparire.
- i client partecipano raramente ai round di addestramento: se il client bizantino non partecipa abbastanza al training da insegnare al modello la backdoor da utilizzare nella seconda fase.
- i dati sono molto eterogenei

## FUBA:

Anche FUBA è un backdoor attack.

Ci sono due tipi di categorie:

1. *Adv-Attacker*: sono i client che iniettano la backdoor nel modello.  
inviando aggiornamenti che insegnano al modello comportamenti malevoli.
2. *Adv-Defender*: sono i client che nascondono la backdoor nel modello.  
inviando aggiornamenti che contrastano quelli degli Adv-Attacker.

Quando finiscono, la backdoor è nascosta.

Gli Adv-Defender chiedono il Federated Unlearning.

Ciò elimina i dati che coprono la backdoor.

rispetto a BadFU:

in BadFU c'era un solo client malevolo, e due dataset diversi.

In FUBA ci sono client differenti, quindi c'è una collaborazione tra più client malevoli.

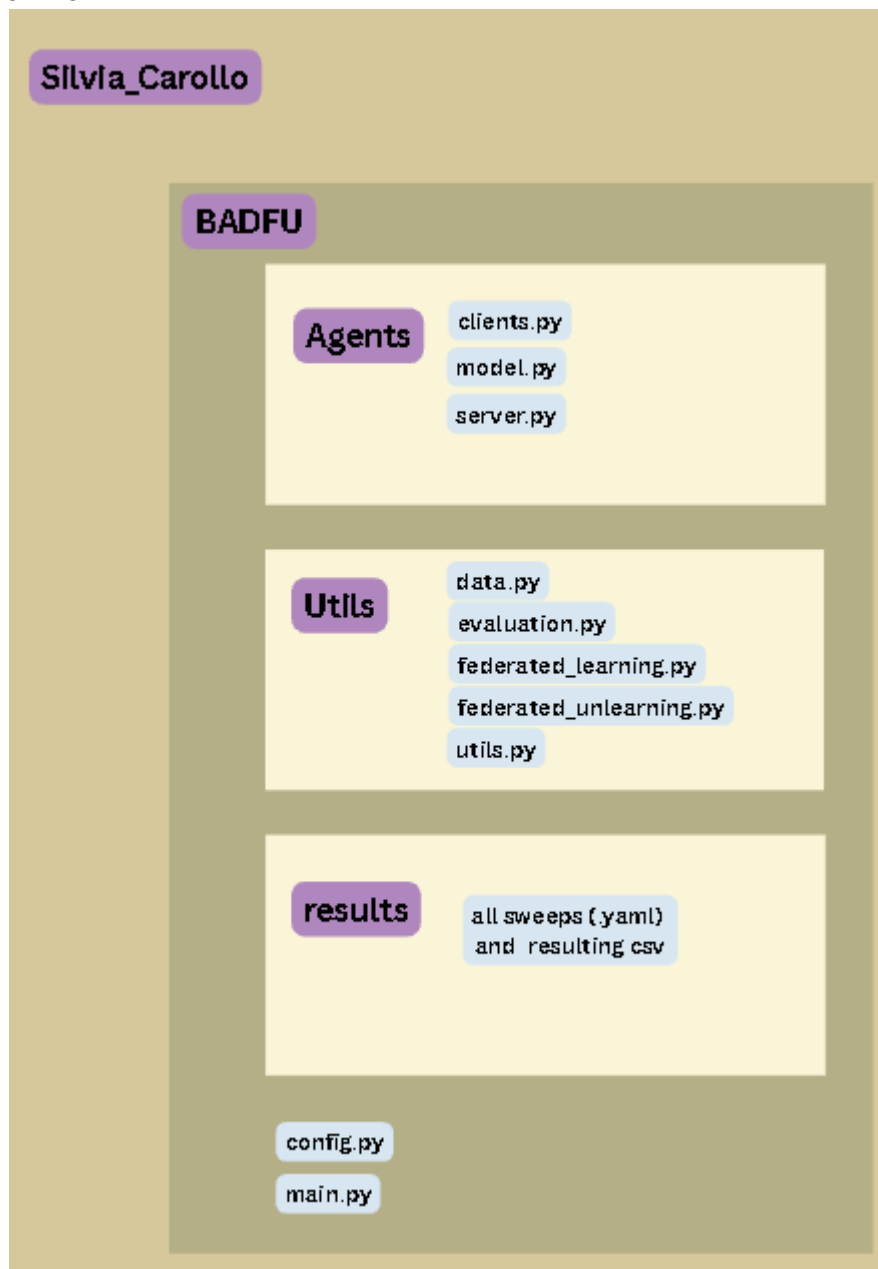
osservazioni:

funziona con diversi algoritmi di FU, mantiene la backdoor nascosta ed elude molti meccanismi di difesa.

# IMPLEMENTAZIONE:

## Struttura del progetto:

Di seguito si farà riferimento al codice della seguente [repository GitHub](#) per mostrare come gli algoritmi sono stati implementati nel codice. Si ritiene utile presentare una guida visiva:



Il progetto si costruisce su tre classi principali:

- **Server**
- **Client**, BadClient
- **BADFU** (il modello)

## Il modello:

Il modello di rete neurale riprende quello del progetto di Ottone Piazzì e possiede tre layers:

- *Input* : prende in input un'immagine in scala dei grigi, tipicamente di 28 x 28 pixels.
- *Layer* : strato denso da 512 unità con attivazione ReLU.
- *Output* : strato di uscita finale (Linear) con numero di uscite pari al numero delle classi (numero di etichette associabili ad un'immagine)

## Gli algoritmi possibili:

Secondo il paper[3], BADFU può essere impiegato sia con diversi metodi di learning che di unlearning.

Si elencano quelli individuati da tale paper con descrizione annessa:

| LEARNING |                                                                                                                                              | UNLEARNING |                                                                                                                                                                                                                                                                                                                                  |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FedAvg   | ogni client svolge il training in locale per un numero fisso di epoche e poi manda i parametri aggiornati al server, che fa una media pesata | Retraining | ri-allena il modello da zero, eliminando il client che ha richiesto l'unlearning                                                                                                                                                                                                                                                 |
| FedSGD   | come il FedAvg, ma aggrega i gradienti locali ad ogni step del training locale, invece che aggregarli alla fine di tutte le epoche.          | FedEraser  | è una tecnica di federated unlearning approssimato. Durante l'addestramento iniziale, i client salvano periodicamente i propri aggiornamenti locali; in fase di rimozione, i client rimasti riallineano tali aggiornamenti per simulare la traiettoria di addestramento senza il client revocato, garantendo un'eliminazione dei |

|         |                                                                                                                                                                                                                                                                                   |      |                                                                                                                                                                                                                                                                                                                                                                        |
|---------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|         |                                                                                                                                                                                                                                                                                   |      | dati rapida ed efficiente senza dover ripartire da zero.                                                                                                                                                                                                                                                                                                               |
| FedProx | <p>è un'estensione di FedAvg, che però incorpora un termine prossimale nella funzione obiettivo locale, con l'obiettivo di gestire l'eterogeneità statistica penalizzando deviazioni significative rispetto al modello di riferimento globale durante l'addestramento locale.</p> | FedU | <p>è un metodo di unlearning basato su approssimazione di funzioni. Il client che richiede l'unlearning stima l'impatto dei dati da rimuovere e toglie tale influenza dal proprio allenamento in locale. Gli altri client contribuiscono a preservare l'utilità del modello globale.</p>                                                                               |
|         |                                                                                                                                                                                                                                                                                   | SIFU | <p>Registra la sensibilità dei client durante l'addestramento e, all'arrivo di una richiesta di rimozione, individua l'epoca in cui la sensibilità del client scende sotto una soglia prestabilita. Da quel punto in poi, inietta rumore per cancellarne l'impronta e prosegue con poche epoche di addestramento finale per recuperare le prestazioni del modello.</p> |

Per il Federated Learning è stato scelto come metodo il **FedAvg**, mentre per l'Unlearning il metodo **FedU**.

## FEDERATED LEARNING (FedAvg):

è basato sull'algoritmo del Federated Averaging che possiamo trovare nel paper [1], ripreso anche da Ottone Piazzì per il learning.

---

**Algorithm 1** FederatedAveraging. The  $K$  clients are indexed by  $k$ ;  $B$  is the local minibatch size,  $E$  is the number of local epochs, and  $\eta$  is the learning rate.

---

**Server executes:**

```
initialize  $w_0$ 
for each round  $t = 1, 2, \dots$  do
   $m \leftarrow \max(C \cdot K, 1)$ 
   $S_t \leftarrow$  (random set of  $m$  clients)
  for each client  $k \in S_t$  in parallel do
     $w_{t+1}^k \leftarrow \text{ClientUpdate}(k, w_t)$ 
   $w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$ 
```

**ClientUpdate( $k, w$ ):** // Run on client  $k$   
 $B \leftarrow$  (split  $\mathcal{P}_k$  into batches of size  $B$ )  
**for** each local epoch  $i$  from 1 to  $E$  **do**  
**for** batch  $b \in B$  **do**  
 $w \leftarrow w - \eta \nabla \ell(w; b)$   
**return**  $w$  to server

---

in Utils/federated\_learning.py

```
##LEARNING##

def federated_learning(model: BADFU, device, clients: list, clients_part = CLIENTS_PART, clients_to_have: list = None, clients_to_avoid: list = None,
    curr_model = copy.deepcopy(model)

    for rnd in range(training_num_rounds):
        client_results = []

        clients_participating = choose_clients(clients, clients_part, clients_to_have, clients_to_avoid)
        print(f"[FEDERATED LEARNING]: clients participating in round {rnd}: {[c.id for c in clients_participating]}")
        for client in clients_participating:
            local_model = copy.deepcopy(curr_model) # each client has its own model to train locally

            state_dict, n_samples, loss = client.train_model(local_model, device)

            client_results.append({state_dict, n_samples, loss})
        #tutti i client hanno fatto il loro training locale, quindi possiamo fare l'update del modello
        curr_model = fedavg(curr_model, client_results)

    return curr_model
```

Il server sceglie randomicamente i client che partecipano e passa a ciascuno di essi una copia del modello in modo che possano lavorare in locale.

Il client restituisce i seguenti parametri che riguardano un modello:

- **state\_dict** = un dizionario PyTorch che mappa ciascuno strato (*layer*) del modello locale ai corrispondenti *pesi e bias aggiornati* dopo l'addestramento sui dati locali (`model.state_dict()`).
- **n\_samples** = numero dei campioni (dati utilizzati dal client)
- **loss** = il *valore finale della funzione di perdita* (es. *Cross-Entropy Loss*) calcolato dal client sui propri dati al termine dell'addestramento locale; serve al server per monitorare l'andamento della convergenza del modello globale

L'ultimo passo del federated learning è il federated averaging.

Il server possiede una lista dei parametri sopra descritti e ne fa una media pesata per poter ottenere i valori finali con cui aggiornare il modello globale.

```
#FedAvg
def fedavg(global_model, client_results):
    #quanti campioni in totale provenienti da TUTTI i client?
    total_n = sum(n for _, n, _ in client_results)
    # client_results possiede triple del tipo (state_dict, n_samples, loss).
    #Stiamo sommando gli n_samples

    agg = OrderedDict() #creiamo una directory ordinata

    for sd, n, _ in client_results: #cicliamo sulle tuple di client_results e prendiamo state_dict ed n_samples
        w = n / total_n # calcoliamo il valore del peso (basato su quanti samples rispetto a quelli complessivi)
        for key in sd:
            val = w * sd[key].float().cpu() #calcoliamo il valore del peso in base agli state_dict
            agg[key] = val if key not in agg else agg[key] + val

    global_model.load_state_dict(agg)

    return global_model
```

in Agents/[server.py](#)

Server side:

```
##LEARNING##
def train(self, clients, device, clients_part = CLIENTS_PART, clients_to_have = None, clients_to_avoid = None):
    self.model = federated_learning(self.model, device, clients, clients_part, clients_to_have, clients_to_avoid)
```

Il server aggiorna il proprio modello attuale dopo il training federato.

in Agents/[clients.py](#)

Client side:

```

def train_model(self, model, device, learning_epochs: int = LEARNING_EPOCHS, training_lr = TRAINING_LR):
    model = model.to(device)
    model.train()

    opt = torch.optim.SGD(model.parameters(), lr=training_lr, momentum=0.9, weight_decay=5e-4)
    loss_fn = nn.CrossEntropyLoss()

    loader = DataLoader(self.data, batch_size = BATCH_SIZE, shuffle = True)# farà BackdoorDataset.__getitem__()

    total_loss = 0.0
    n_batches = 0

    for _ in range(learning_epochs):
        for imgs, labels in loader:
            imgs, labels = imgs.to(device), labels.to(device)
            opt.zero_grad()
            loss = loss_fn(model(imgs), labels)
            loss.backward()

            opt.step()
            total_loss += loss.item()
            n_batches += 1

    model.to("cpu")
    return model.state_dict(), len(self.data), total_loss / max(n_batches, 1)

```

Il client lavora sulla copia del modello che gli è stata assegnata.

Carica un BATCH del proprio dataset e svolge un numero fisso di epoche in cui:

per ciascuna immagine a cui è associata un'etichetta

1. calcola l'output del modello sull'immagine e la loss rispetto all'etichetta corretta (CrossEntropyLoss)
2. calcola i gradienti tramite backpropagation (loss.backward)
3. aggiorna i pesi dell'optimizer SGD (opt.step)

Una volta terminate le epoche restituisce i parametri.

## FEDERATED UNLEARNING (FEDU):

Si è scelto il metodo FEDU per poter osservare gli effetti di BADFU in un ambiente con molteplici client.

Questo metodo è basato sull'algoritmo FEDU via User-Side Influence Approximation Forgetting, dal paper [3] qui sotto riportato:

---

**Algorithm 1:** FedU via User-Side Influence Approximation Forgetting.

---

**Input:**  $K$  is the number of users;  $E$  is the number of local epochs,  $\eta$  is the learning rate.

- 1 **FL server executes:**
- 2 Reinitialize the global model  $\mathcal{M}_u^{t=0}(\theta) \leftarrow \mathcal{M}(\theta)$ , and send it to all users;
- 3 **User executes:**
- 4 **for all users in parallel do**
- 5     **for**  $t = 0, 1, \dots, T$  **do**
- 6         **if is unlearned user then**
- 7             IAF-U( $k_u, \mathcal{M}_u^t(\theta), E, D_{k_u}^e, D_{k_u}^r$ );
- 8             Uploads  $\mathcal{M}_{k_u}^t$  to server and receives  $\mathcal{M}_u^{t+1}$ ;
- 9             **if** Verification( $\mathcal{M}_u^{t+1}, D_{k_u}^e$ )==Yes **then**  
              change to normal user;
- 10         **else**
- 11             NormalTraining( $k_c, \mathcal{M}_u^t(\theta), E, D_{k_c}$ );
- 12             Uploads  $\mathcal{M}_{k_c}^t$  to server and receives  $\mathcal{M}_u^{t+1}$ ;

Il server inizializza l'unlearning, ovvero, segnala ai suoi client di dover svolgere l'unlearning, passa quindi a ciascuno di essi una copia locale del modello attuale su cui poter lavorare.

Tutti i client che hanno richiesto l'unlearning utilizzano il metodo IAF-U per trainare il modello, mentre quelli che non l'hanno richiesto lo allenano con il normal training.

In seguito all'aggiornamento i singoli client mandano i parametri finali al server.

## IAF-U

### I gradienti e la loss:

In questa funzione entrano in gioco i seguenti componenti, ovvero, i due gradienti e le loss associate:

1.  $g_{IAF}$ :

è la direzione di oblio (influence approximation forgetting).

Misura l'influenza esercitata dai dati da cancellare sul modello globale corrente.

La sua funzione di calcolo si basa sulla seguente espressione presente nel paper[3] :

$$\Delta\theta \simeq \mathcal{I}^{(1)}(D^e) = \frac{1}{n-m} H_{\theta^*}^{-1} \sum_{z_j \in D^e} \ell(z_j, \theta^*),$$

In essa compare una matrice hessiana inversa.

Dato che tale computazione sarebbe stata troppo costosa computazionalmente, si è preferita fare una stima di tipo LISSA.

```
def iaf_direction(model, criterion, erased_imgs, erased_labels, kept_imgs, kept_labels, n_total, m,
    fu_depth: int = FU_DEPTH,
    fu_damping: float = FU_DAMPING,
    fu_scale: float = FU_SCALE,
    iaf_scale: float = IAF_SCALE):
    """
    Li af = arg min_theta* 1/(n-m) * sum(loss(z,theta))
    attenzione, la minimizzazione della media va fatta fuori.
    #si fa loss.backward() per ottenere il gradiente
    """
    params = trainable_params(model)
    theta0 = flat([p.detach() for p in params])

    model.zero_grad()
    out = model(erased_imgs)
    # erased_imgs ora contiene TUTTI gli m campioni cancellati (vedi DataLoader
    # in IAF_U), quindi mean*count e' gia' la somma ESATTA richiesta dall'Eq. 6
    # -- niente piu' bisogno di estrapolare moltiplicando per m
    loss_erased_sum = criterion(out, erased_labels) * erased_imgs.size(0)
    grads_erased = torch.autograd.grad(loss_erased_sum, params, create_graph=True)
    v = flat(grads_erased).detach()

    t = min(fu_depth, kept_imgs.size(0)) # totale campioni usati per la ricorsione LISSA
    lissa_batch_size = min(LISSA_BATCH_SIZE, t) # quanti campioni per passo di ricorsione
    remaining_samples = [
        (kept_imgs[i:i + lissa_batch_size], kept_labels[i:i + lissa_batch_size])
        for i in range(0, t, lissa_batch_size)
    ]
    # es. t=20, lissa_batch_size=5 -> 4 passi di ricorsione invece di 20,
    # stessa informazione (20 campioni), 5x meno forward/backward

    ihvp = estimate_inverse_hvp(model, criterion, remaining_samples, v, damping=fu_damping, scale=fu_scale)

    theta_target = theta0 + 1/(n_total-m) * iaf_scale * ihvp # scala trattata come iperparametro, non più 1/(n-m) fisso

    pseudo_grad = theta0 - theta_target
    return pseudo_grad
```

\* Le funzioni `hessian_vector_product`, `estimate_inverse_hvp` permettono tale calcolo. Si trovano in `Utils/federated_unlearning.py`.

## 2. $L_{IAF}$ : loss di oblio.

$$\text{IAF Loss: } \mathcal{L}_{IAF} = \arg \min_{\theta^*} \frac{1}{n-m} \sum_{z \in D \setminus D_{k_u}^e} \ell(z, \theta).$$

## 3. $g_{UP}$ :

è la direzione di preservazione dell'utilità (utility preservation).

Serve ad evitare il degrado delle prestazioni del modello sui dati mantenuti (kept).

In questo caso utilizzare l'hessiana non è necessaria, poiché è un gradiente del primo ordine calcolato a ogni epoca sui mini-batch del dataset dei dati mantenuti.

```
def up_grad(model, criterion, kept_imgs, kept_labels):
    """
        L_up
        theta* = arg min_theta l(D, theta) {(1/n) [sum_z_belong_D (l(z, theta)) ]}
    """
    params = trainable_params(model)
    model.zero_grad()
    out = model(kept_imgs)
    loss = criterion(out, kept_labels)
    grads = torch.autograd.grad(loss, params)
    return flat(grads).detach(), loss.item()
```

4.  $L_{UP}$ : loss di preservazione dell'utilità.

$$\theta^* = \arg \min_{\theta} \ell(D, \theta) = \arg \min_{\theta} \frac{1}{n} \sum_{z_i \in D} \ell(z_i, \theta)$$

**L'algoritmo IAF-U:**

```
13 IAF-U( $k_u, \mathcal{M}_u^t(\theta), D_{k_u}^e, D_{k_u}^r$ ):    ▷ Run on the
    unlearned user  $C_{k_u}$ 
14 for  $E$  epochs do
15     Sample a minibatch,  $\{z_i : (x_j, y_j)\}_{j=1}^m$ , from erased
        dataset  $D_{k_u}^e$ ;
16     Sample a minibatch,  $\{z_j : (x_i, y_i)\}_{i=1}^m$ , from
        remaining dataset  $D_{k_u}^r$ ;
17     Calculate the influence approximation forgetting
        loss  $\mathcal{L}_{IAF}$  according to Eq. (7);
18     Calculate the utility preservation loss  $\mathcal{L}_{UP}$  according
        to Eq. (1);
19     Calculate  $d$  using Eq. (10) based on  $\mathcal{L}_{IAF}$  and  $\mathcal{L}_{UP}$ ,
        and update the model as:  $\theta \leftarrow \theta + \eta d$ 
```

La funzione IAF viene svolta dal client che ha richiesto l'unlearning in locale : per un certo numero fisso di epoche.

Utils/federated\_unlearning.py

```

def IAF_U(model, data, client_id: int, indexes_to_erase: list, device,
          iaf_fu_epochs: int = IAF_FU_EPOCHS,
          iaf_scale: float = IAF_SCALE,
          lr_update_fu: float = LR_UPDATE_FU,
          fu_depth: int = FU_DEPTH,
          fu_damping: float = FU_DAMPING,
          fu_scale: float = FU_SCALE,
          max_d_norm: float = MAX_D_NORM,
          fixed_lambda: float = FIXED_LAMBDA):
    model = model.to(device)
    n_total = len(data)
    m = len(indexes_to_erase)
    # dataset dei campioni da dimenticare
    erased_data = Subset(data, indexes_to_erase)
    # otteniamo il dataset dei campioni da mantenere
    erase_set = set(indexes_to_erase)

    retain_indices = [
        i for i in range(len(data))
        if i not in erase_set
    ]

    kept_data = Subset(data, retain_indices)

    # dataloaders
    # per gli erased usiamo TUTTI gli m campioni in un solo batch: ci serve
    # la somma ESATTA dei gradienti (Eq. 6), non una stima rumorosa estrapolata
    # da un sotto-campione (che con m grande amplifica troppo il rumore)
    erased_loader = DataLoader(erased_data, batch_size=len(erased_data), shuffle=True, drop_last=False)
    kept_loader = DataLoader(kept_data, batch_size=BATCH_SIZE, shuffle=True, drop_last=False)

    kept_batches = cycle(kept_loader)

    criterion = nn.CrossEntropyLoss()

    # --- direzione IAF: calcolata UNA SOLA VOLTA, al punto originale theta* ---
    # Eq. 6 e' una correzione valida localmente, vicino al punto in cui e'
    # stata stimata l'Hessiana. Ricalcolarla ad ogni epoca dopo che il modello

```

Come specificato nello pseudocodice sopra, ad ogni epoca, si devono estrarre un mini-batch di dati da cancellare e uno dai dati da mantenere.

Per questo motivo si partiziona il dataset locale nei due sottoinsiemi:

- *erased data* = campioni da cancellare
- *kept data* = campioni da conservare

Si inizializzano i loader di dati per poterli estrarre in maniera efficiente.

```

# si e' gia' spostato la rende sempre meno affidabile: il valore cresce
# epoca dopo epoca (l'abbiamo visto nel log) e il combinatore MGDA finisce
# per darle peso ~0 perche' la giudica troppo "in conflitto" con grad_UP.
# Congelandola qui evitiamo la deriva, e risparmiamo anche il costo di
# ristimare l'Hessiana ad ogni epoca.
erased_imgs, erased_labels = next(iter(erased_loader))
erased_imgs = erased_imgs.to(device)
erased_labels = erased_labels.to(device)

kept_imgs0, kept_labels0 = next(kept_batches)
kept_imgs0 = kept_imgs0.to(device)
kept_labels0 = kept_labels0.to(device)

g_iaf = iaf_direction(model, criterion, erased_imgs, erased_labels, kept_imgs0, kept_labels0, n_total, m, fu_depth=fu_depth, fu_damping=fu_damping, fu_scale=fu_scale, iaf_scale=iaf_scale)
print(f"[client {client_id}] |pseudo_grad IAF| (fissa) = {torch.norm(g_iaf).item():.6f}")
if wandb.run is not None:
    wandb.log({f"unlearning/client_{client_id}/iaf_norm_fixed": torch.norm(g_iaf).item()})

total_loss = 0.0

for epoch in range(iaf_fu_epochs):
    kept_imgs, kept_labels = next(kept_batches)
    kept_imgs = kept_imgs.to(device)
    kept_labels = kept_labels.to(device)

    # utility preservation loss (questa si ricalcola ogni epoca: e' un
    # gradiente di training normale, stabile per costruzione)
    g_up, up_loss = up_grad(model, criterion, kept_imgs, kept_labels)

    total_loss += up_loss

    d, lam, beta = fixed_combine(g_iaf, g_up, fixed_lambda) # niente più MGDA: peso garantito a g_iaf

    # clip sulla norma di d: rete di sicurezza aggiuntiva
    d_norm = torch.norm(d)
    if d_norm > max_d_norm:
        d = d * (max_d_norm / (d_norm + 1e-12))

    # solo per debug: stampa/logga la prima, l'ultima, e una ogni 10 epoche

```

Il gradiente  $g_{iaf}$  viene calcolato prima del loop per evitare la deriva numerica causata dal ricalcolo su parametri non più ottimali.

Ad ogni epoca si calcolano invece  $g_{up}$  e la  $up\_loss$ .

L'obiettivo è in realtà quello di combinare le due loss secondo la seguente espressione:

$$d_{k_u, t} = -(\lambda \nabla \mathcal{L}_{IAF}(\theta_{k_u, t}) + \beta \nabla \mathcal{L}_{UP}(\theta_{k_u, t}))$$

where  $\lambda + \beta = 1$ ,

In questo progetto  $\lambda$  è fissa a 0.5 e si applica un clipping alla combinazione per evitare la crescita instabile.

Tale combinazione serve a bilanciare le due forze.

```

# solo per debug: stampa/logga la prima, l'ultima, e una ogni 10 epoche
# (logga su wandb OGNI epoca con FU_EPOCHS alto e' quello che stava
# rallentando tutto: ogni wandb.log() e' una chiamata di rete, con
# FU_EPOCHS=300 su 2 client erano 600 chiamate)
if epoch == 0 or epoch == iaf_fu_epochs - 1 or epoch % 10 == 0:
    print(f"[client {client_id}] epoch {epoch} | |grad UP| = {torch.norm(g_up).item():.6f} | "
          f"|d| = {torch.norm(d).item():.6f} | lambda = {lam.item():.3f}")

    if wandb.run is not None:
        wandb.log({
            f"unlearning/client_{client_id}/epoch": epoch,
            f"unlearning/client_{client_id}/up_norm": torch.norm(g_up).item(),
            f"unlearning/client_{client_id}/d_norm": torch.norm(d).item(),
            f"unlearning/client_{client_id}/lambda": lam.item(),
            f"unlearning/client_{client_id}/up_loss": up_loss,
        })

    apply_update(model, d, lr_update_fu)

# if verified(model):
#     print(f"[client {client_id}] verifica superata all'epoca {epoch}, arresto unlearning")
#     break

avg_loss = total_loss / max(iaf_fu_epochs, 1)

return model, avg_loss

```

Infine si aggiornano i parametri del modello locale (apply\_update)  
 Si restituisce la loss media e il modello aggiornato (con unlearning).

## normal training:

Viene svolta dai client coinvolti nell'unlearning che però non l'hanno richiesto:

```

20 NormalTraining( $k_c, \mathcal{M}_u^t(\theta), D_{k_c}$ ):  $\triangleright$  Run on
    normal FL user  $C_{k_c}$ 
21 for  $E$  epochs do
22     Sample minibatch  $\{z_i : (x_i, y_i)\}_{i=1}^m$  from local  $D_{k_c}$ ;
23     Calculate the training loss according to Eq. (1);
24     Update:  $\theta \leftarrow \theta - \eta \nabla_{\theta} \ell(\cdot, \theta)$ 
25 FL server executes:
26 Update unlearned global model, aggregating local
    models from all users as Eq. (11):
     $\mathcal{M}_u^{t+1}(\theta) \leftarrow \frac{1}{K} \sum_{k=1}^K \mathcal{M}_k^t(\theta_k)$ ;
27 Return  $\mathcal{M}_u(\theta)$ ;

```

da Utils/federated\_unlearning.py

```
def normal_training(model, data, device, n_fu_epochs: int = N_FU_EPOCHS, lr_update_fu: float = LR_UPDATE_FU):
    model = model.to(device)
    model.train()

    criterion = nn.CrossEntropyLoss()

    data_loader = DataLoader(data, batch_size=BATCH_SIZE, shuffle=True, drop_last=False)
    optimizer = torch.optim.SGD(model.parameters(), lr=lr_update_fu, momentum=0.9, weight_decay=5e-4)

    total_loss = 0.0
    n_batches = 0

    for epoch in range(n_fu_epochs):
        for imgs, labels in data_loader:
            imgs, labels = imgs.to(device), labels.to(device)
            optimizer.zero_grad()
            loss = criterion(model(imgs), labels)
            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            n_batches += 1

    avg_loss = total_loss / max(n_batches, 1)
    return model, avg_loss
```

In questo caso si utilizza l'addestramento locale del modello sui dati del client tramite *Stochastic Gradient Descent (SGD)*, per un numero di epoche pari a *n\_fu\_epochs*.

Il dataset locale viene suddiviso in mini-batch e processato secondo il seguente flusso:

1. si azzerano i gradienti per cancellare i gradienti accumulati nei passaggi precedenti. (`optimizer.zero_grad()`)
2. si calcola l'output del modello sui dati del batch e si valuta l'errore mediante la funzione di costo Cross-Entropy.
3. si calcola la loss di entropia incrociata sui dati del batch
4. si calcolano i gradienti della loss rispetto ai parametri del modello tramite backpropagation
5. aggiornano i pesi del modello

Alla fine si calcola e restituisce il valore medio della loss su tutti i batch totali processati insieme al modello aggiornato.

nel file Agents/[server.py](#) :

```

def unlearning(self, model, device, clients_part = CLIENTS_PART, **unlearn_kwargs):
    results = []

    #####always_present = request.ids
    always_present = self.requests.get_all_ids()
    print(f"clients that requested unlearning: {always_present}\n")

    unlearning_clients = choose_clients(self.clients, clients_part, always_present, None)
    print(f"clients that will participate to unlearning: {[client.id for client in unlearning_clients]}\n")

    for client in unlearning_clients:
        local_model = copy.deepcopy(model)
        results.append(client.unlearn_model(local_model, self.requests, device, **unlearn_kwargs ))

    self.model = fedavg(model, results)

def request_unlearning(self, client_id : int, samples_to_erase: Subset):
    self.requests.add(client_id, samples_to_erase)

```

Il server calcola i client che partecipano all'unlearning e richiede loro di fare l'unlearning passando loro una copia del modello.

Inserisci i risultati in una lista e poi ne fa il federated averaging.

Il server permette inoltre ai client di formulare una richiesta di unlearning.

nel file Agents/[clients.py](#) :

```

def unlearn_model(self, model, requests: list[Request], device,
                  iaf_fu_epochs: int = IAF_FU_EPOCHS,
                  iaf_scale: float = IAF_SCALE,
                  lr_update_fu: float = LR_UPDATE_FU,
                  fu_depth: int = FU_DEPTH,
                  fu_damping: float = FU_DAMPING,
                  fu_scale: float = FU_SCALE,
                  max_d_norm: float = MAX_D_NORM,
                  fixed_lambda: float = FIXED_LAMBDA,
                  n_fu_epochs: int = N_FU_EPOCHS):
    request = requests.get(self.id)
    if request is not None:
        client_id, indexes_to_erase = request
        model, loss = IAF_U(model, self.data, client_id, indexes_to_erase, device,
                             iaf_fu_epochs=iaf_fu_epochs, iaf_scale=iaf_scale,
                             lr_update_fu=lr_update_fu, fu_depth=fu_depth,
                             fu_damping=fu_damping, fu_scale=fu_scale,
                             max_d_norm=max_d_norm, fixed_lambda=fixed_lambda)
        requests.remove(client_id)
    else:
        # model, data_loader, optimizer, criterion, local_epochs: int, device
        model, loss = normal_training(model, self.data, device, n_fu_epochs=n_fu_epochs, lr_update_fu=lr_update_fu)

    model.to("cpu")
    return (model.state_dict(), len(self.data), loss)

def request_unlearning(self, server, samples_to_erase: float = SAMPLES_TO_ERASE):
    # Request unlearning from the server
    # casual data to erase (to simulate a normal user)
    total_len = len(self.data)
    num_samples_to_erase = int(total_len * samples_to_erase)
    indexes_to_erase = random.sample(range(0, total_len), num_samples_to_erase)
    self.erased_indices = indexes_to_erase # PER DEBUG

    server.request_unlearning(self.id, indexes_to_erase)

```

- il client viene coinvolto nell'unlearning. Ci sono due possibilità:
  - ha formulato una richiesta -> utilizza l'algoritmo IAF
  - non ha formulato una richiesta -> utilizza l'algoritmo di normal training
- Il client può formulare una richiesta di unlearning al server.

## I CLIENT:

In questo modello i client contribuiscono all'addestramento del modello e possono richiedere l'unlearning.

L'addestramento viene fatto in locale su una copia del modello, utilizzando il proprio set di dati.

I dati non escono mai dal locale, seguendo i principi del Federated Unlearning.

Al termine di ogni allenamento il client manda al server i parametri.

```
class Client:
    def __init__(self, id, data):
        self.id = id
        self.data = data # i dati sono già un Subset

    def train_model(self, model, device, learning_epochs: int = LEARNING_EPOCHS, training_lr = TRAINING_LR):
        model = model.to(device)
        model.train()

        opt = torch.optim.SGD(model.parameters(), lr=training_lr, momentum=0.9, weight_decay=5e-4)
        loss_fn = nn.CrossEntropyLoss()

        loader = DataLoader(self.data, batch_size = BATCH_SIZE, shuffle = True)# farà BackdoorDataset.__getitem__()

        total_loss = 0.0
        n_batches = 0

        for _ in range(learning_epochs):
            for imgs, labels in loader:
                imgs, labels = imgs.to(device), labels.to(device)
                opt.zero_grad()
                loss = loss_fn(model(imgs), labels)
                loss.backward()

                opt.step()
                total_loss += loss.item()
                n_batches += 1

        model.to("cpu")
        return model.state_dict(), len(self.data), total_loss / max(n_batches, 1)
```

```

def unlearn_model(self, model, requests: list[Request], device,
                 iaf_fu_epochs: int = IAF_FU_EPOCHS,
                 iaf_scale: float = IAF_SCALE,
                 lr_update_fu: float = LR_UPDATE_FU,
                 fu_depth: int = FU_DEPTH,
                 fu_damping: float = FU_DAMPING,
                 fu_scale: float = FU_SCALE,
                 max_d_norm: float = MAX_D_NORM,
                 fixed_lambda: float = FIXED_LAMBDA,
                 n_fu_epochs: int = N_FU_EPOCHS):
    request = requests.get(self.id)
    if request is not None:
        client_id, indexes_to_erase = request
        model, loss = IAF_U(model, self.data, client_id, indexes_to_erase, device,
                           iaf_fu_epochs=iaf_fu_epochs, iaf_scale=iaf_scale,
                           lr_update_fu=lr_update_fu, fu_depth=fu_depth,
                           fu_damping=fu_damping, fu_scale=fu_scale,
                           max_d_norm=max_d_norm, fixed_lambda=fixed_lambda)
        requests.remove(client_id)
    else:
        # model, data_loader, optimizer, criterion, local_epochs: int, device
        model, loss = normal_training(model, self.data, device, n_fu_epochs=n_fu_epochs, lr_update_fu=lr_update_fu)

    model.to("cpu")
    return (model.state_dict(), len(self.data), loss)

def request_unlearning(self, server, samples_to_erase: float = SAMPLES_TO_ERASE):
    # Request unlearning from the server
    # casual data to erase (to simulate a normal user)
    total_len = len(self.data)
    num_samples_to_erase = int(total_len * samples_to_erase)
    indexes_to_erase = random.sample(range(0, total_len), num_samples_to_erase)
    self.erased_indices = indexes_to_erase # PER DEBUG

    server.request_unlearning(self.id, indexes_to_erase)

```

## I BadClient sono i client bizantini che attuano l'attacco.

Essi possiedono due tipi di dato seguendo quanto anzidetto sull'attacco BADFU:

- *avvelenati* = immagine con trigger con etichetta target
- *camuffati* = immagine senza trigger con etichetta originale

Il BadClient estende la classe Client, e fa l'override della richiesta di unlearning.

Quando il Client normale richiede l'unlearning, vengono selezionati dei dati casuali da cancellare, mentre, quando lo richiede il BadClient vengono selezionati i dati camuffati.

```

class BadClient(Client):
    def __init__(self, id, data, clean_img_per: float = CLEAN_IMG, poison_img_per: float = POISON_IMG):
        super().__init__(id, data)
        self.data = BackdoorDataset(data, clean_img_per, poison_img_per)

    def request_unlearning(self, server):
        # Request unlearning from the server
        # we erase the camo
        self.erased_indices = self.data.get_camo_indices()
        server.request_unlearning(self.id, self.erased_indices)

```

Si utilizza una classe di aiuto per il dataset che a partire dai dati iniziali, genera in automatico quelli camuffati e quelli avvelenati:

```

class BackdoorDataset(Dataset):
    def __init__(self, original_dataset, clean_img_per: float, poison_img_per: float):
        self.original_dataset = original_dataset
        total_len = len(original_dataset)

        self.num_clean = int(total_len * clean_img_per)
        self.num_poison = int(total_len * poison_img_per)
        self.num_camo = total_len - self.num_clean - self.num_poison
        self.poison_start = self.num_clean
        self.camo_start = self.num_clean + self.num_poison

    def __len__(self):
        return len(self.original_dataset)

    def __getitem__(self, index):
        #normale training
        if index < self.poison_start:
            image, label = self.original_dataset[index]
            return image, label

        #poisoned data : trigger + etichetta target
        elif index < self.camo_start :
            image, _ = self.original_dataset[index]
            image = image.clone()

            image[0, -TRIGGER_SIZE: , -TRIGGER_SIZE: ] = TRIGGER_VAL

            return image, TARGET_LABEL

        #camouflage data: no trigger + etichetta originale
        else:
            image, label = self.original_dataset[index]
            image = image.clone()

            image[0, -TRIGGER_SIZE: , -TRIGGER_SIZE: ] = TRIGGER_VAL

            return image, label

    def get_camo_indices(self): #per poter fare dopo l'unlearning
        return list(range(self.camo_start, len(self.original_dataset)))

```

# LA RICERCA:

## Valutazione:

Nella valutazione si usano i seguenti criteri, consigliati dal paper [3]:

- **Benign Accuracy (ACC)** = è la percentuale delle immagini indovinate con successo dal modello.

*da cui deriva:*

**Accuracy Drop** = la percentuale che costituisce la differenza tra la ACC finale e quella iniziale. Permette di capire quanto diminuisce l'accuracy.

- **Attack Success Rate (ASR)** = è l'attack success rate, ovvero, quante immagini con trigger il modello identifica con l'etichetta decisa dall'attaccante.

*da cui deriva:*

**ASR Increase** = la percentuale che costituisce la differenza tra la ASR finale e quella iniziale. Permette di capire quanto aumenta l'ASR.

In Utils/[evaluation.py](#) si trovano le seguenti funzioni:

```

def evaluate_accuracy_and_ASR(model, test_data, device, stage: str = "eval"):
    """
    Calcola contemporaneamente:
    - Accuracy sul test set pulito
    - Attack Success Rate (ASR) sul test set con trigger.
    //Bisogna eliminare quelle che hanno la stessa target label!
    """

    loader = DataLoader(test_data, batch_size=BATCH_SIZE, shuffle=False)

    model.to(device).eval()

    correct = 0
    total = 0

    successful = 0
    total_backdoor = 0

    with torch.no_grad():

        for imgs, labels in loader:

            imgs = imgs.to(device)
            labels = labels.to(device)

            # =====
            # 1. ACCURACY SU IMMAGINI PULITE
            # =====

            outputs = model(imgs)
            predictions = outputs.argmax(1)

            correct += (predictions == labels).sum().item()
            total += labels.size(0)

```

```

# =====
# 2. ASR SU IMMAGINI CON TRIGGER
# =====

mask = labels != TARGET_LABEL
if mask.any():
    poisoned_imgs = imgs[mask].clone()
    poisoned_imgs[:, 0, -TRIGGER_SIZE:, -TRIGGER_SIZE:] = TRIGGER_VAL

    poisoned_predictions = model(poisoned_imgs).argmax(dim=1)

    successful += (poisoned_predictions == TARGET_LABEL).sum().item()

    total_backdoor += mask.sum().item()

model.to("cpu")

accuracy = correct / max(total,1)
asr = successful / total_backdoor if total_backdoor > 0 else 0.0

if wandb.run is not None:
    wandb.log({f"{stage}/accuracy":accuracy, f"{stage}/asr":asr})

return accuracy, asr

```

permette di calcolare ASR e Accuracy.

Per la valutazione dell'ASR, si eliminano le immagini che hanno in partenza come etichetta il valore scelto dall'attaccante per evitare falsi positivi.

```

def compare_models(old_model, new_model):
    total_diff = 0.0
    total_old = 0.0

    old_params = dict(old_model.named_parameters())
    new_params = dict(new_model.named_parameters())

    for name in old_params:
        old_p = old_params[name].detach().float()
        new_p = new_params[name].detach().float()

        total_diff += torch.sum((new_p - old_p) ** 2).item()
        total_old += torch.sum(old_p ** 2).item()

    total_diff = total_diff ** 0.5
    total_old = total_old ** 0.5
    ratio = (total_diff / total_old) if total_old > 0 else 0.0

    if wandb.run is not None:
        wandb.log({
            "model_change/absolute": total_diff,
            "model_change/pre_norm": total_old,
            "model_change/ratio": ratio
        })

    return {
        "absolute": total_diff,
        "pre_norm": total_old,
        "ratio": ratio}

```

Questa funzione permette di calcolare ASR Increase e Accuracy drop, comparando il modello iniziale e quello finale.

## Dataset:

- **MNIST** = è un dataset che contiene 70000 cifre scritte a mano in bianco e nero. Tale dataset possiede 60000 immagini di allenamento e 10000 immagini di test. Ogni immagine (28 x 28) contiene una singola cifra scritta a mano.

nel file Utils/[data.py](#) :

Questa è la funzione che permette di caricare il dataset:

si preparano un dataset per il training, che verrà utilizzato dai clients, e uno per il testing, che verrà utilizzato dal server.

```
def load_mnist():
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.1307,), (0.3081,)),
    ])
    train = datasets.MNIST(root="data", train=True, download=True, transform=transform)
    test = datasets.MNIST(root="data", train=False, download=True, transform=transform)
    return train, test
```

In questa simulazione lo si distribuisce cosicché i client abbiano tutti lo stesso numero di campioni di dati (IID).

```
def partition_iid(dataset, num_clients): # returns a dictionary with id_client :
    indices = np.random.permutation(len(dataset))
    chunks = np.array_split(indices, num_clients)
    return {
        i: chunks[i].tolist()
        for i in range(num_clients)
    }
```

## La simulazione:

Il main() simula la parte decisionale degli agenti che ne prendono parte.

```
def main():
    """
    //////////////////////////////////////////////////////////////////////////////////////////////////////////////////
    INITIALIZATION
    //////////////////////////////////////////////////////////////////////////////////////////////////////////////////
    """
    config = {
        "num_clients": NUM_CLIENTS,
        "perc_bad_clients": PERC_BAD_CLIENTS,

        "learning_num_rounds": TRAINING_NUM_ROUNDS,
        "learning_epochs": LEARNING_EPOCHS,
        "training_lr": TRAINING_LR,

        "normal_training_fu_epochs": N_FU_EPOCHS,
        "iaf_fu_epochs": IAF_FU_EPOCHS,
        "iaf_scale": IAF_SCALE,
        "lr_update_fu": LR_UPDATE_FU,

        "fu_depth": FU_DEPTH,
        "fu_damping": FU_DAMPING,
        "fu_scale": FU_SCALE,
        "max_d_norm": MAX_D_NORM,

        "clean_img_per": CLEAN_IMG,
        "poison_img_per": POISON_IMG,

        "samples_to_erase": SAMPLES_TO_ERASE,
        "unlearn_clients_part": UNLEARN_CLIENTS_PART,

        "type_of_clients_who_request_unlearning": TYPE_OF_CLIENTS_WHO_REQUEST_UNLEARNING
    }

    with wandb.init(project="badfu_attack", config=config) as run:

        cfg = wandb.config

        initial_config(seed=cfg.seed)
```

Si inizializza il progetto in modo tale da prepararlo agli sweep di Wandb.

Questo significa preparare dei parametri che possano essere stabiliti dalla configurazione di Wandb.

```

##### INITIALIZATION

#global load of data for simulation
good_train, good_test = load_mnist()
print("loaded mnist")

#create model
model = BADFU()
print("instantiated global model")

#create server
server = Server(model, good_test)
print("instantiated server")

##### CLIENT PARTITION

#create clients
#each client must have its own data
#WE slice for ALL the clients NUM_GOOD_CLIENTS + NUM_BAD_CLIENTS, and the bad clients will modify their slice as they please

client_indices = partition_iid(good_train, cfg.num_clients) # { client id : indexes of data for client }
print("instantiated clients")

num_bad_clients = int(cfg.num_clients * cfg.perc_bad_clients)
num_good_clients = cfg.num_clients - num_bad_clients

print("GOOD CLIENTS: ", num_good_clients)
print("BAD CLIENTS: ", num_bad_clients)

for i in range(cfg.num_clients):
    train_subset = Subset(good_train, client_indices[i])
    if i < num_good_clients:
        server.add_client(Client(i, train_subset))
    else:
        server.add_client(BadClient(i, train_subset, clean_img_per=cfg.clean_img_per, poison_img_per=cfg.poison_img_per))

print("partitioned data")

device = initial_config()
print("device : ", device)

```

Si carica il dataset e lo si distribuisce tra i vari client in modo che ognuno di loro possieda il proprio set.

Si inizializzano il server e i clients che possono essere buoni o bizantini.  
Il loro numero, le loro percentuali, vengono stabilite dalla configurazione di WandB.

Il server non sarà mai in grado di vedere i dati dei singoli clients.

```

"""
////////////////////////////////////
                        TRAINING
////////////////////////////////////
"""

print("\n===== TRAINING =====\n")

train_clients_part = get_learning_clients_percentage(cfg.num_clients)
# train model with federated learning

print(
    f"Training client participation: "
    f"{train_clients_part*100:.1f}%"
)

server.train(server.clients, device, train_clients_part) #training iniziale, partecipano tutti

"""
////////////////////////////////////
                        PRE-UNLEARNING
////////////////////////////////////
"""

print("\n===== PRE-UNLEARNING =====\n")

#saving pre-unlearning model to get metrics later
old_model = copy.deepcopy(server.model)#to check later

# check how many corrects over total, ecc...
valuate.evaluate_accuracy_and_ASR(server.model, server.test_data, device, stage="pre_unlearning")

```

Una volta stabilito quanti clients parteciperanno all'addestramento.

Il server inizializza il training, ovvero, fornisce ai singoli clients una copia del modello attuale, e riceve i parametri aggiornati da questi ultimi.

In seguito svolge il Federated Averaging a partire da essi e aggiorna il modello attuale.

Si fa una valutazione *prima* dell'unlearning e se ne salvano i risultati.

```

////////////////////////////////////
UNLEARNING
////////////////////////////////////
"""

match cfg.type_of_clients_who_request_unlearning:
    case 0: #bad clients
        unlearning_req_clients = server.get_bad_clients()
    case 1: #good clients
        unlearning_req_clients = server.get_good_clients()
    case 2: #all clients
        unlearning_req_clients = server.clients

for client in unlearning_req_clients:
    client.request_unlearning(server)

server.evaluate_unlearning_sets(device,stage="pre_unlearning")

print("\n===== UNLEARNING =====\n")
server.unlearning(server.model, device, cfg.unlearn_clients_part,
    iaf_fu_epochs=cfg.iaf_fu_epochs,
    iaf_scale=cfg.iaf_scale,
    lr_update_fu=cfg.lr_update_fu,
    fu_depth=cfg.fu_depth,
    fu_damping=cfg.fu_damping,
    fu_scale=cfg.fu_scale,
    max_d_norm=cfg.max_d_norm,
    n_fu_epochs=cfg.normal_training_fu_epochs) #unlearning, partecipano solo i client che hanno richiesto l'unlearning

server.evaluate_unlearning_sets(device,stage="post_unlearning")

print(f"unlearning completed.\n")

```

Si sceglie il tipo di clients che richiederanno l'unlearning (questa scelta serve solo agli scopi della ricerca).

I clients mandano una richiesta di unlearning al server.

Una volta che il server ha ricevuto le richieste, inizia l'unlearning, con i parametri stabiliti.

```

""" ////////////////////////////////////////////////////
            METRICS CHECK
//////////////////////////////////////////////////
"""
print("\n===== POST-UNLEARNING =====\n")

# check metrics now
valuate.evaluate_accuracy_and_ASR(server.model, server.test_data, device, stage="post_unlearning")
valuate.compare_models(old_model, server.model)

# =====
# DERIVED METRICS
# =====

pre_acc = run.summary.get(
    "pre_unlearning/accuracy"
)

post_acc = run.summary.get(
    "post_unlearning/accuracy"
)

pre_asr = run.summary.get(
    "pre_unlearning/asr"
)

post_asr = run.summary.get(
    "post_unlearning/asr"
)

if pre_acc is not None and post_acc is not None:
    wandb.summary["accuracy_drop"] = (
        pre_acc - post_acc
    )

if pre_asr is not None and post_asr is not None:
    wandb.summary["asr_increase"] = (
        post_asr - pre_asr
    )

```

Vengono quindi calcolati accuracy e ASR in seguito all'unlearning e si fa una comparazione con quelli precedenti.

Si ottengono così anche accuracy drop e ASR increase.

## Gli esperimenti:

### Hyper Parameters :

si fissano come consigliati dal paper [3]:

- BATCH\_SIZE = 32
- si usa la discesa stocastica del gradiente (SGD)
- learning rate = 0.01

### 1) *benign control*:

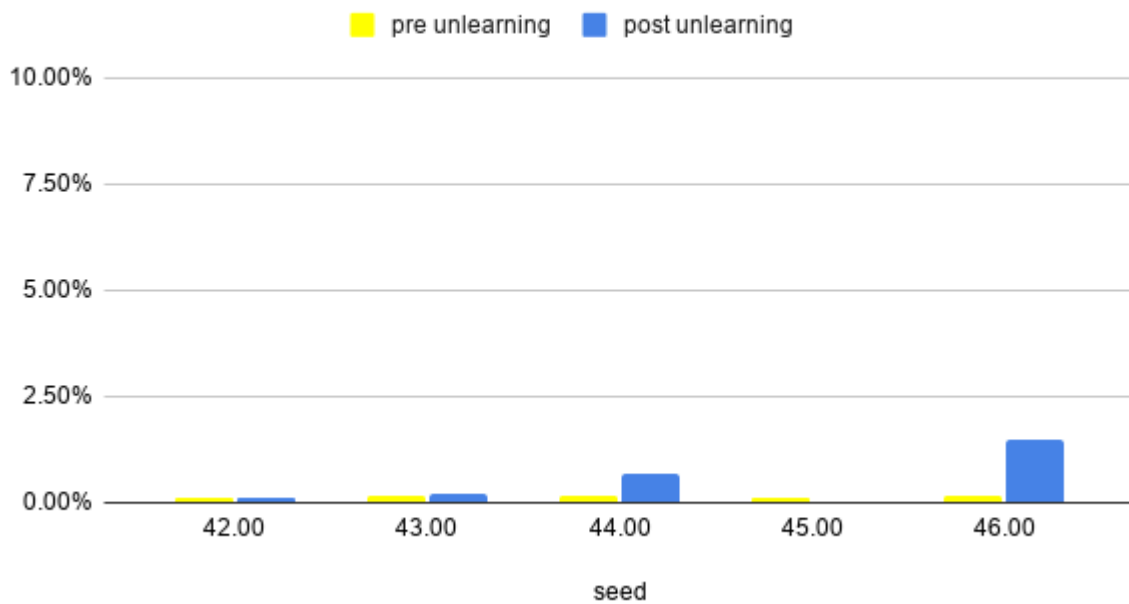
Si è fatto un primo controllo preliminare per osservare il forgetting dei soli good client, per eliminare la possibilità di catastrophic forgetting.

In questa analisi è stato fatto variare il “seed” per poter osservare il modello con randomicità differenti e in tutti e cinque i casi si è notato un ASR così piccolo da poter essere considerato rumore (inferiore allo 0.025, ovvero 2.25%) e un accuracy rate alto che corrisponde allo scenario in cui non viene inserita alcuna backdoor.

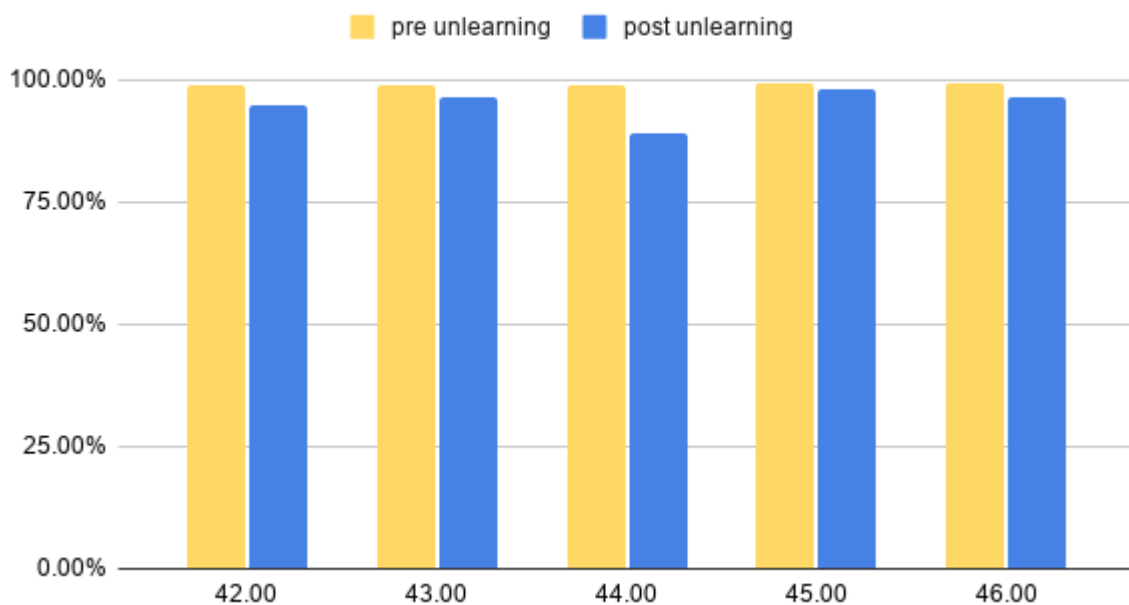
Nota:

sono stati utilizzati i seed 42, 43, 44, 45, 46

## benign control : ASR



## ACCURACY



Si è concluso che il forgetting fatto da client normali non rovina il modello, né porta ad un aumento di successo dell'attacco .

Attenzione: in seguito a questo esperimento, si è ritenuto di poter procedere con l'osservazione dei soli client bizantini per cercare di capire in che modo essi sono più efficaci.

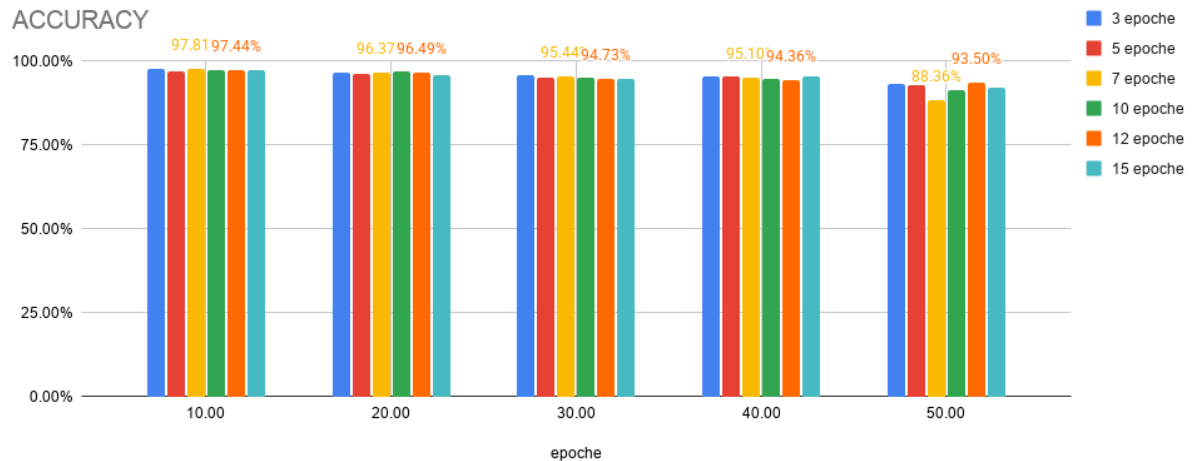
## 2) iaf fu epochs:

si osservano l'accuracy e l'ASR al variare delle epoche dell'*untraining* per trovare una configurazione stabile ma efficace.

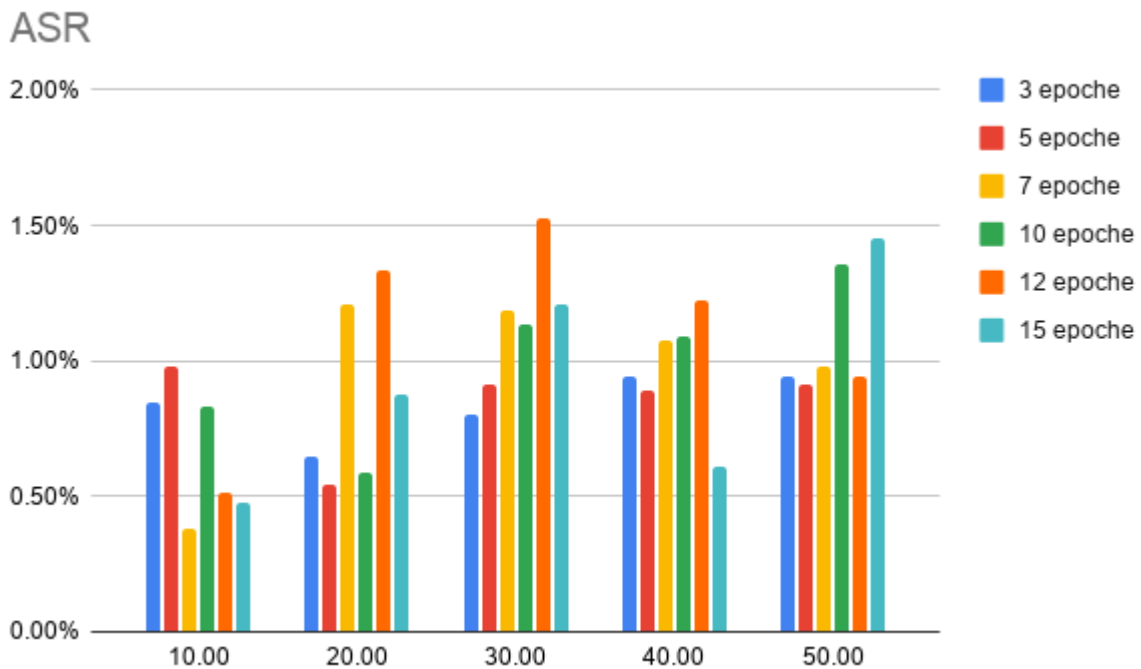
### 2.1 (sweep e2\_num\_clients.yaml, e2\_iaf fu\_epochs\_num\_clients):

si sono fatti variare sia il numero dei clients totali che il numero delle epoche.

La percentuale dei bad clients è in questo caso fissa al 20%.



L'ACCURACY rimane molto alta, a riprova che il modello non è soggetto a Catastrophic forgetting.



l'ASR post unlearning rimane molto bassa (sotto il 2%), pertanto l'attacco in questo caso non sembra avere successo.

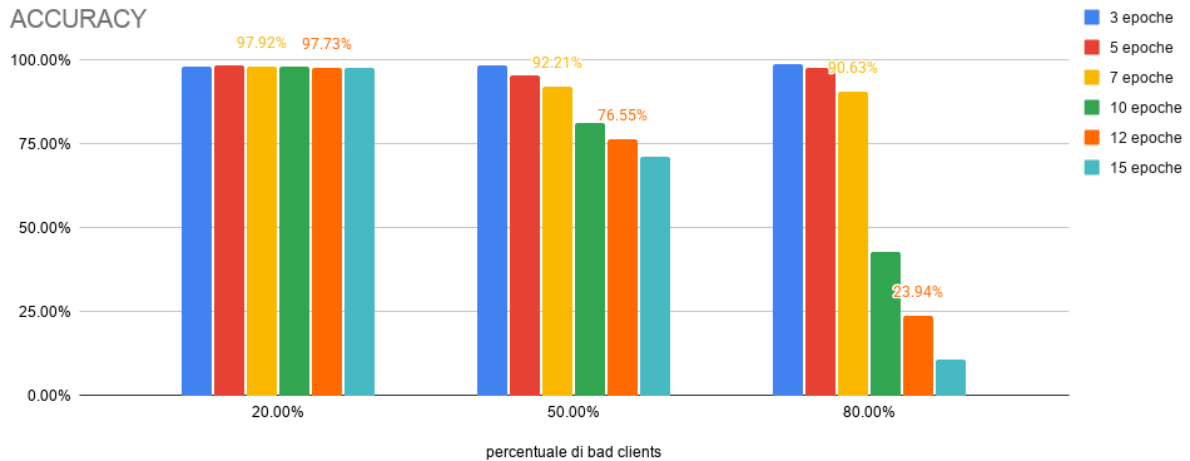
I risultati sono stati inconcludenti.

Si conclude questo esperimento con l'ipotesi che il numero dei clients totali non influenzi così tanto l'ASR, mentre le epoche sì, dato che si notano dei cambiamenti relativamente più grandi tra le 7 e le 15 epoche.

2.2 (sweep\_e2\_more\_bad.yaml, e2\_iaf\_fu\_epochs\_more\_bad\_clients.csv):

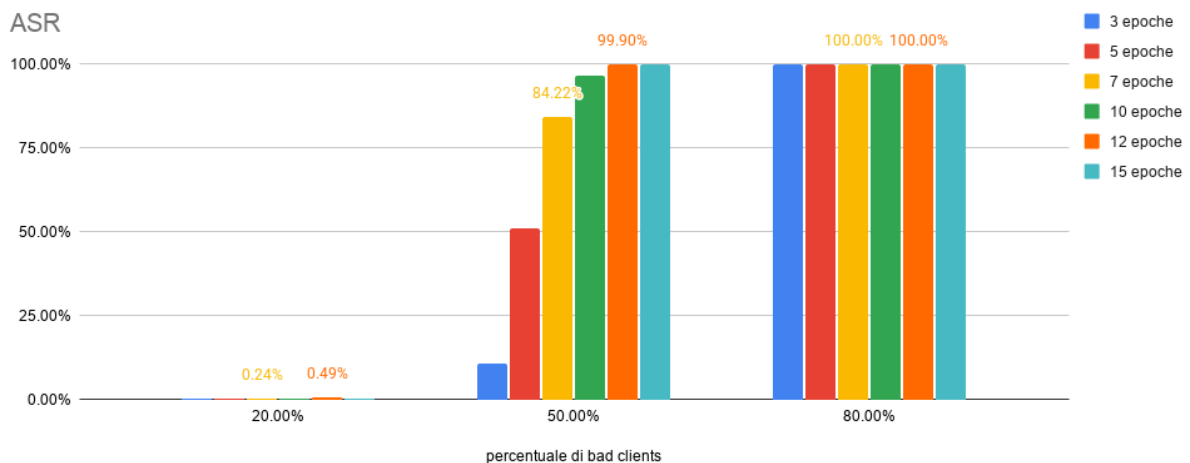
Questo esperimento fa variare la percentuale dei bad clients e delle epoche per capire se era tale percentuale a rendere inefficace l'attacco nell'esperimento 2.1 (erano solo il 20% del totale).

Si osservano l'accuracy e l'ASR post unlearning:



L'accuracy crolla sensibilmente nel caso in cui i clients siano oltre il 50% e le epoche siano maggiori o uguali a 7.

Si suppone di creare in questo modo un unlearning troppo aggressivo che porti pertanto al Catastrophic Forgetting.



Grazie all'ASR possiamo capire che l'attacco ha successo con almeno un 50% di bad clients presenti nella rete e con almeno 5 epoche di unlearning.

Considerando che l'accuracy crolla sensibilmente con l'80% dei client bizantini nella rete, con almeno 7 epoche.

Questo scenario sembra molto irrealistico, in quanto la maggior parte dei client della rete federata dovrebbero essere bizantini.

Si ipotizza che l'ASR vada al 100% a causa della distruzione del modello e non perché l'attacco sia efficace.

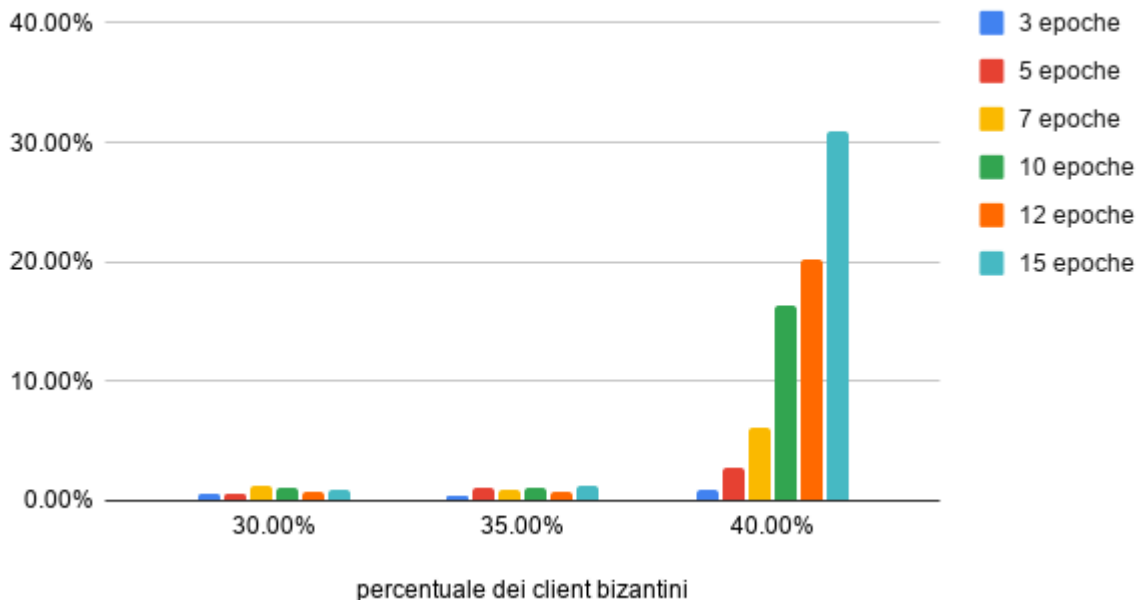
### 2.3 (sweep\_e2\_pinpoint\_bad.yaml,e2\_iat\_fu\_epochs\_pinpoint\_bad) :

Per trovare la combinazione migliore a questo punto si controllano delle percentuali di client bizantini comprese tra il 20%, a cui corrisponde (secondo quanto visto nell'esperimento 2.2) un ASR non efficace, e il 50%, a cui corrisponde invece un aumento dell'ASR all'aumentare delle epoche.

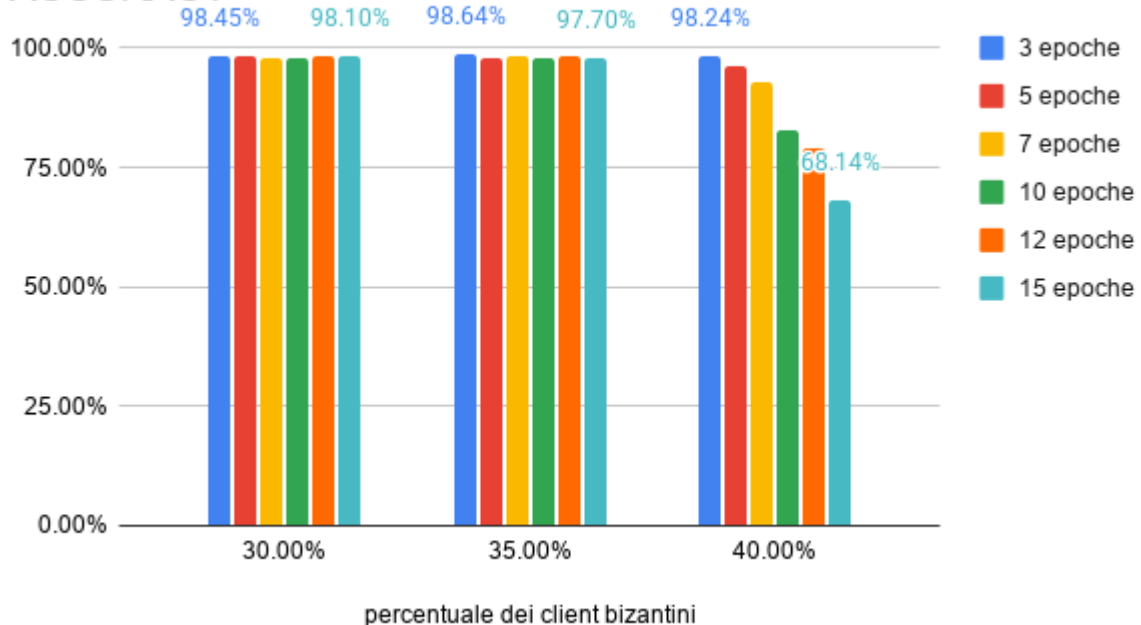
Pertanto si fanno variare le seguenti percentuali di bad clients : 30%, 35% e 40%.

Si osservano quindi ,oltre l'accuracy e l'ASR, il crollo dell'accuracy (accuracy drop) e l'aumento dell'ASR (ASR increase) che comparano i valori pre e post unlearning dell'accuracy e dell'ASR.

### ACCURACY DROP

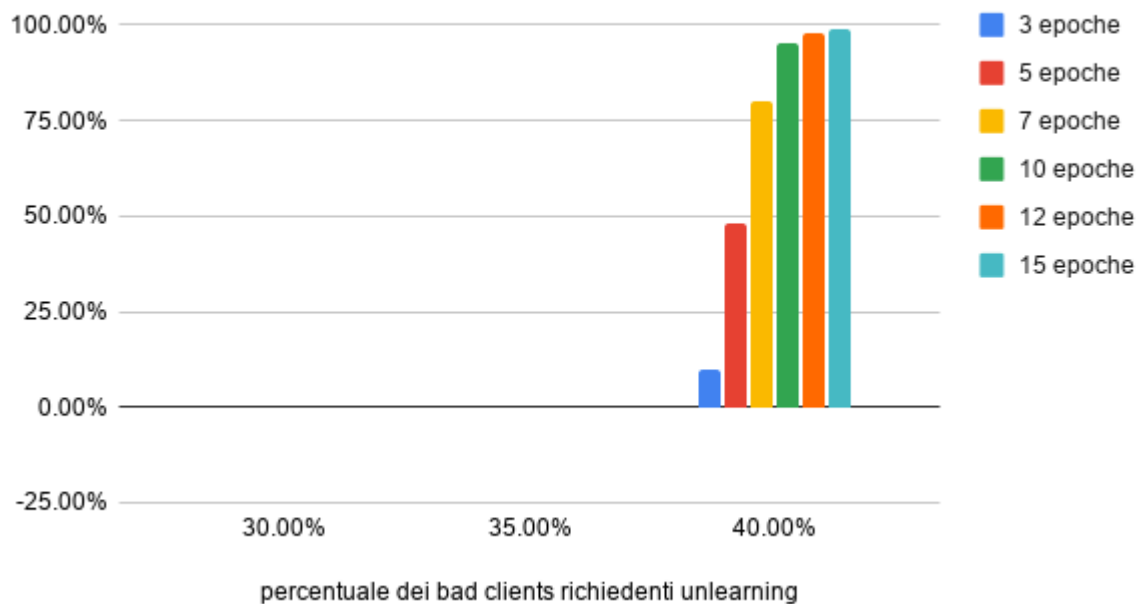


### ACCURACY



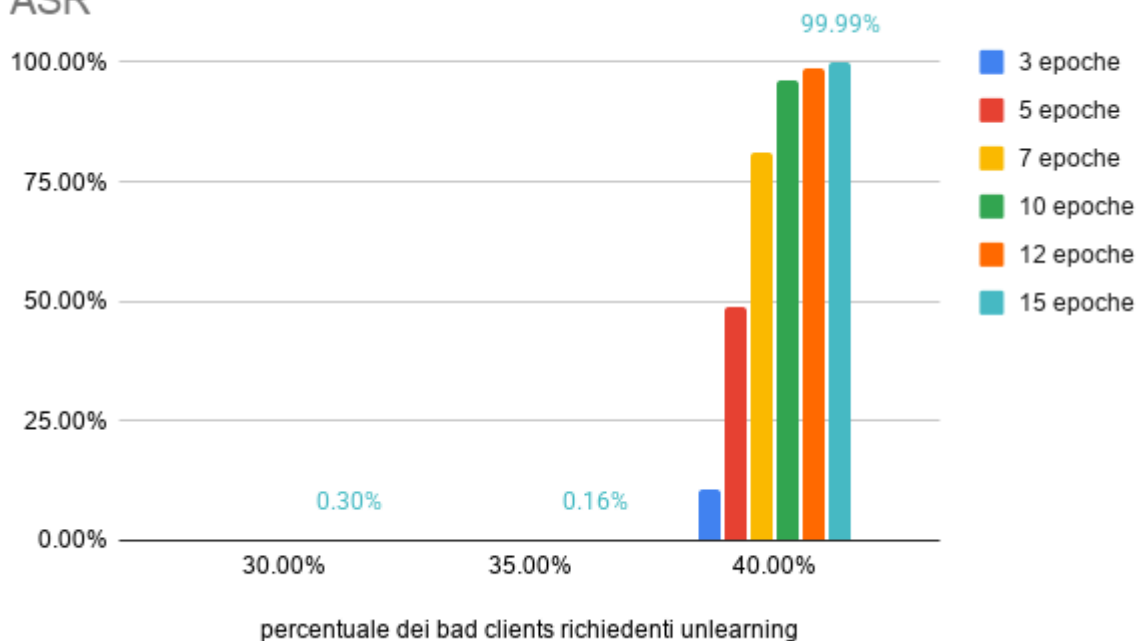
Si osserva un aumento della perdita di accuratezza non accettabile col 40% dei bad clients, a partire dalle 12 epoche.

## ASR INCREASE



In contrasto, il successo dell'attacco aumenta significativamente a partire dalle 5 epoche, con il 40% dei bad clients.

## ASR



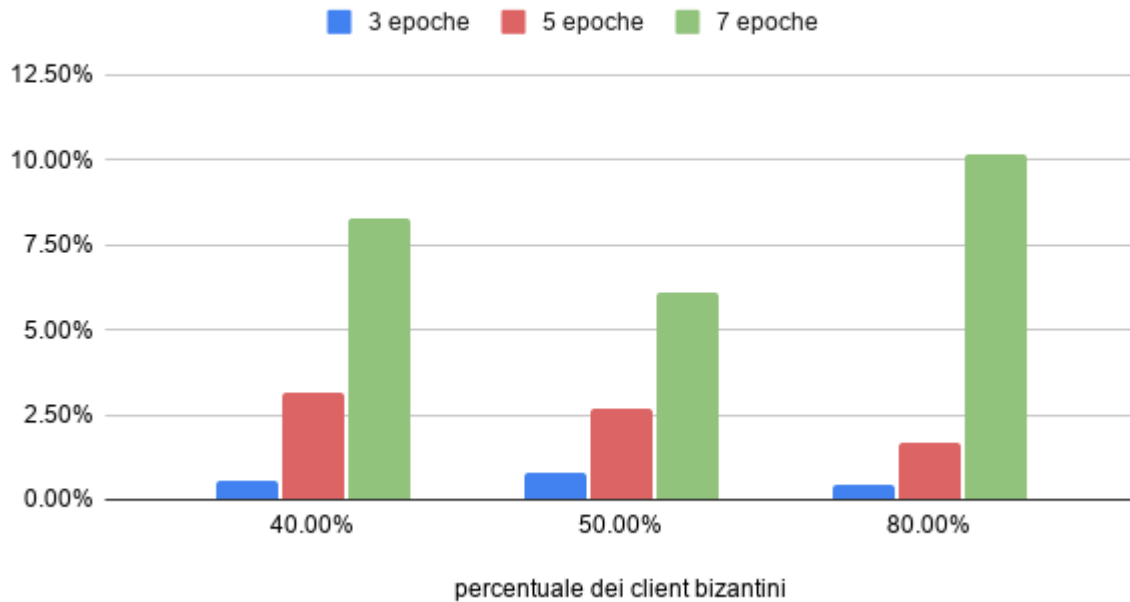
L'ASR dalle 12 epoche in poi è molto alto. Si attribuisce questo risultato al Catastrophic Forgetting del modello.

Si deduce che il miglior compromesso è una percentuale del 40% di client bizantini sul totale, con un numero di epoche di unlearning (iaf) tra le 5 e le 10.

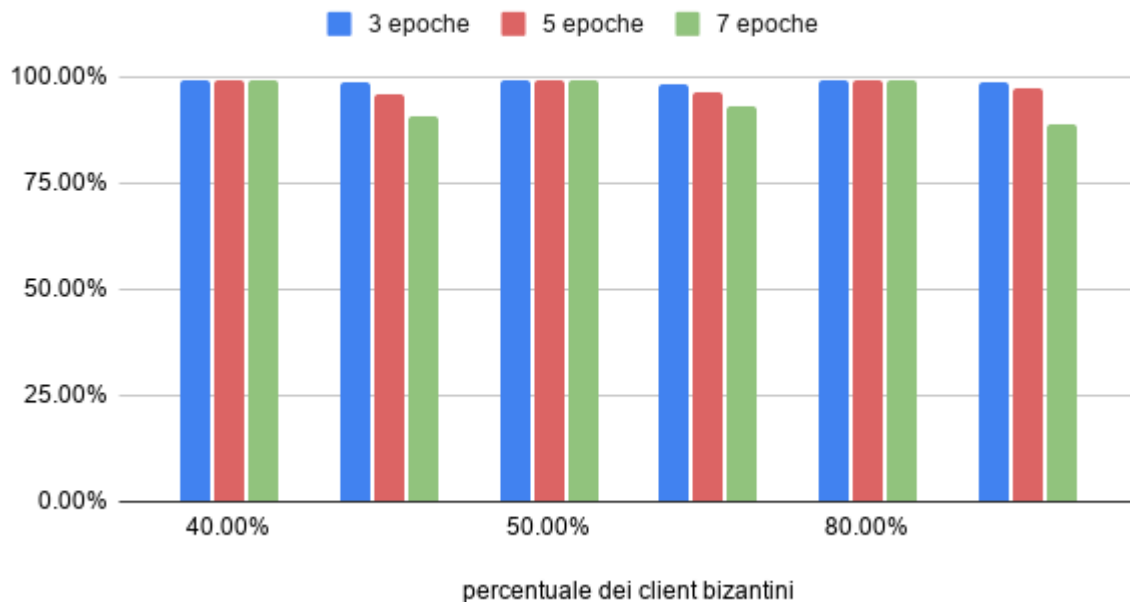
#### 2.4 (sweep e2 find best, e2 iaf fu epochs find best.csv) :

Si osservano le corrispondenze bad clients ed epoche che presentavano i migliori risultati osservati nei precedenti esperimenti, al fine di confrontarli .

### ACCURACY DROP

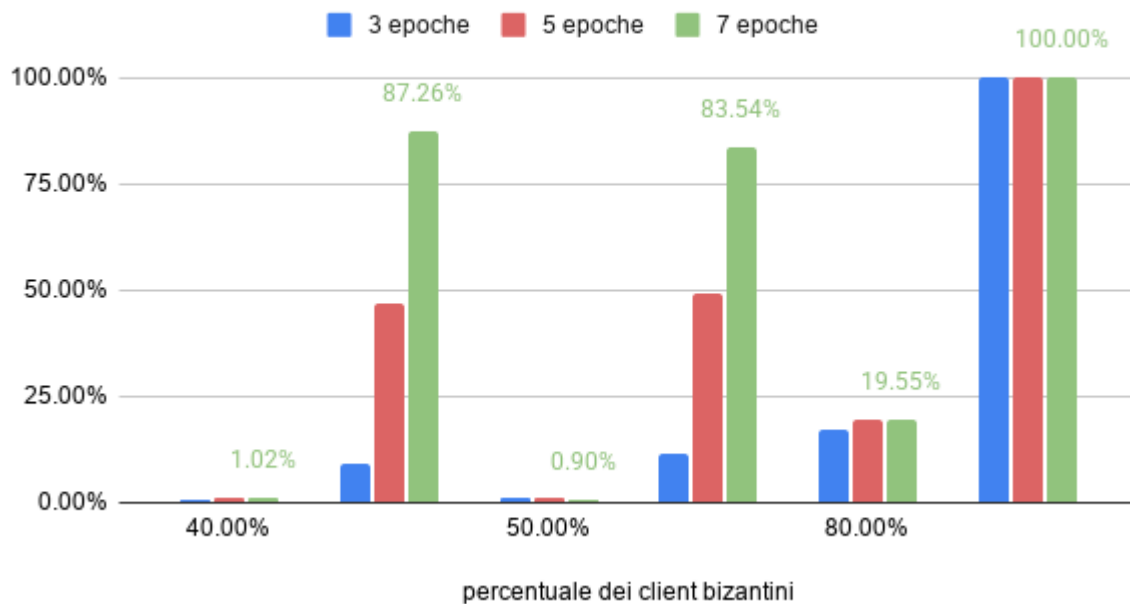


### ACCURACY PRE E POST



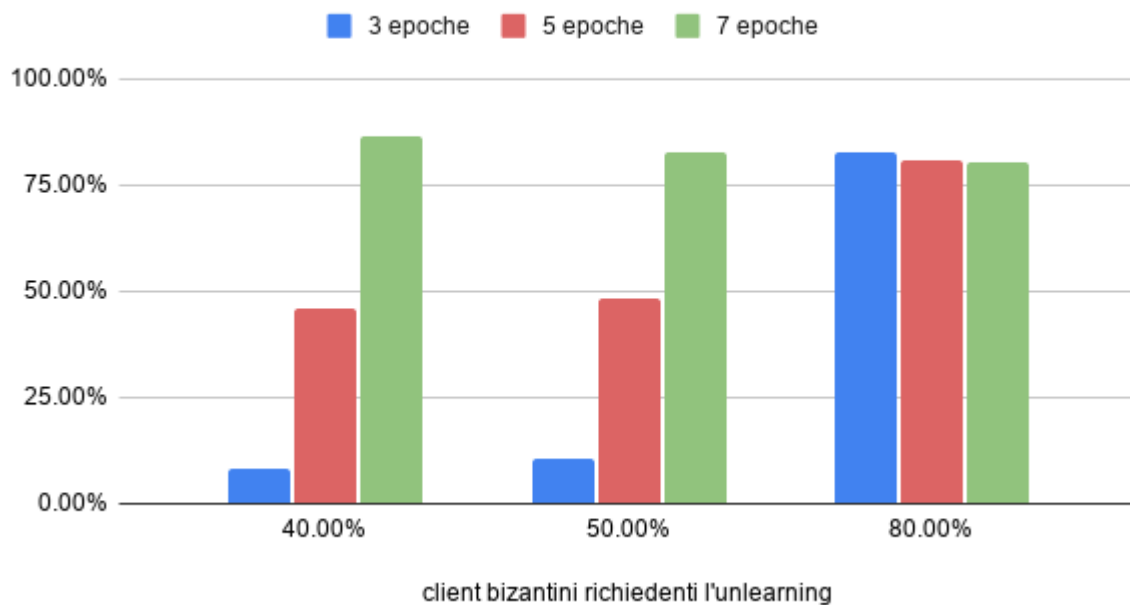
Sebbene non si arrivi mai a distruggere il modello (Catastrophic Forgetting), le configurazioni più instabili sembrano essere quelle con più epoche di unlearning.

## ASR PRE E POST



Con queste configurazioni l'attacco non ha sempre successo.

## ASR INCREASE



L'ASR ha più successo all'aumentare delle epoche e dei clients, ma non si può ignorare la perdita di accuracy.

In questo modo si sono stabiliti i seguenti valori per gli esperimenti successivi:

*epoche di unlearning (iaf) : 7*

*percentuale dei bad client : 40%*

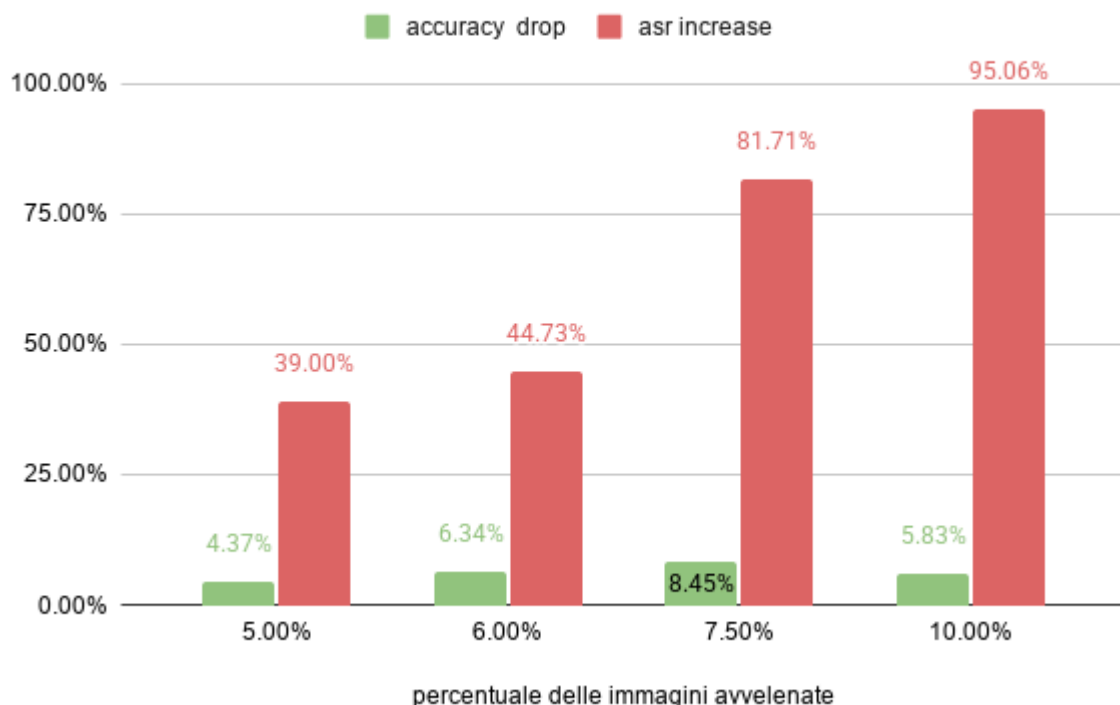
Sebbene la configurazione più stabile ed efficace risulti essere quella con l'80% di client bizantini e 3 epoche, si è scelta quest'ultima perché si ritiene difficile nella realtà riuscire ad immettere in una rete così tanti client bizantini.

Inoltre, si ritiene più utile osservare uno scenario con il 'minimo' necessario per far riuscire l'attacco al meglio.

3) *poison image percentage* (e3\_poison\_ratio.csv, sweep\_e3.yaml):

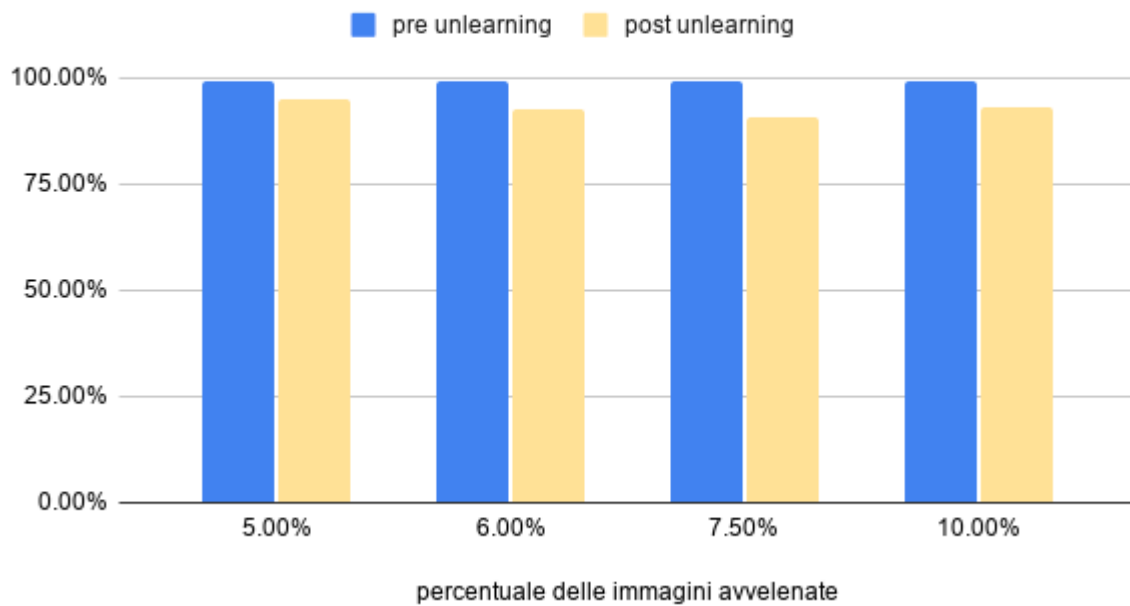
Con questo esperimento ci si propone di osservare l'influenza che ha la percentuale delle immagini avvelenate sull'ASR e l'Accuracy.

## ACCURACY DROP e ASR INCREASE

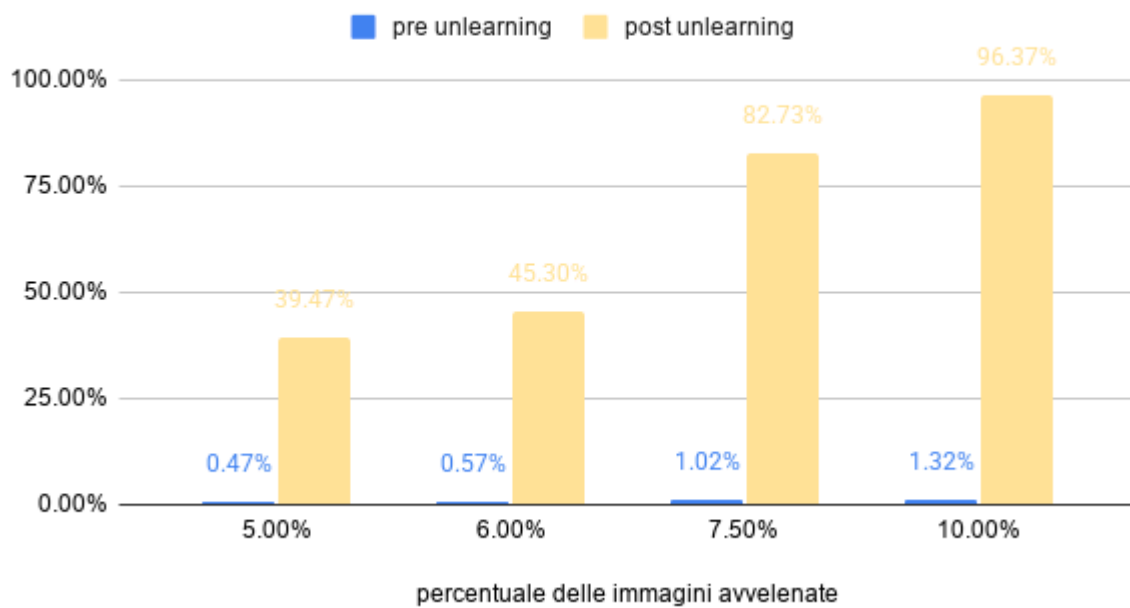


Al crescere della percentuale delle immagini avvelenate si nota un forte aumento del successo dell'attacco, in particolare dopo la soglia del 6% l'ASR inizia a crescere molto. Con altrettanta soddisfazione si può notare che l'accuracy, sebbene diminuisca, rimane comunque abbastanza stabile.

## ACCURACY



## ASR



Possiamo quindi dire che l'attacco ha maggior successo con valori vicini al 7.50%, senza il rischio di Catastrophic Forgetting.

Per questo motivo d'ora in poi si fisserà il valore della percentuale delle immagini avvelenate:

*poison\_img\_per = 7.5%*

4) *unlearn clients part* (e5\_unlearn\_clients\_part, sweep\_e5.yaml):

al variare della percentuale dei clients che richiedono l'unlearning sul totale dei client, si osserva la variazione dell'accuracy e dell'ASR dopo l'unlearning.

Per come è costruito l'unlearning, i clients che richiedono l'unlearning sono solo bizantini, ma anche altri partecipano ad esso se non bastano per raggiungere il numero richiesto dalla percentuale.

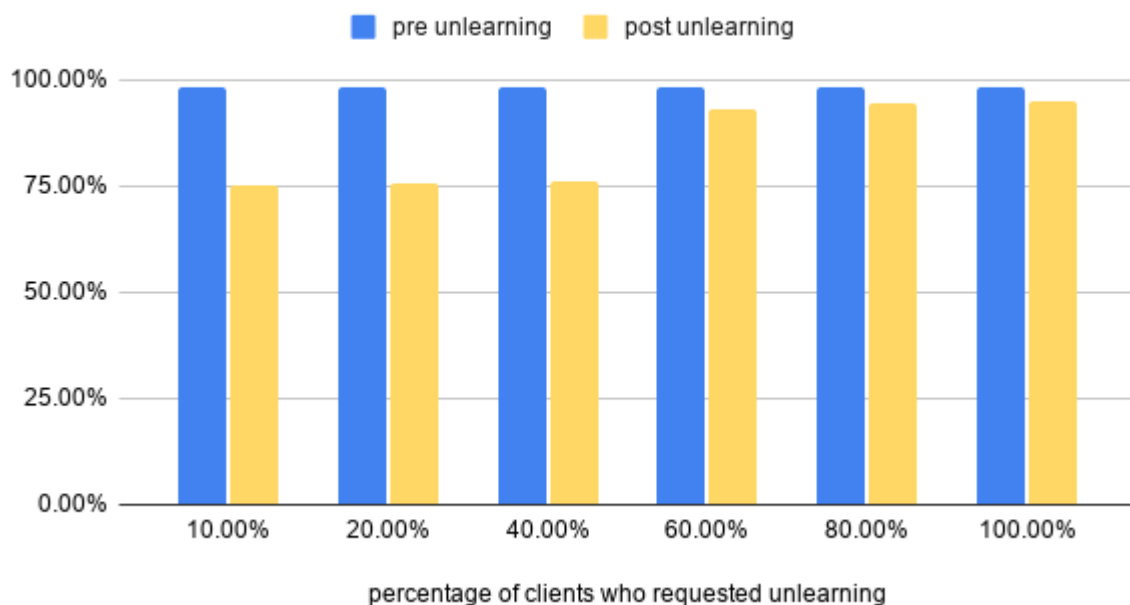
I clients bizantini svolgono l'unlearning IAF mentre gli altri il normale training.

Aumentare l'*unlearn\_clients\_part* significa aumentare il numero di client non bizantini che partecipano all'unlearning.

Difatto, un *unlearn\_clients\_part* corrisponde ai soli client bizantini che richiedono l'unlearning, eliminando il loro effetto contrastante.

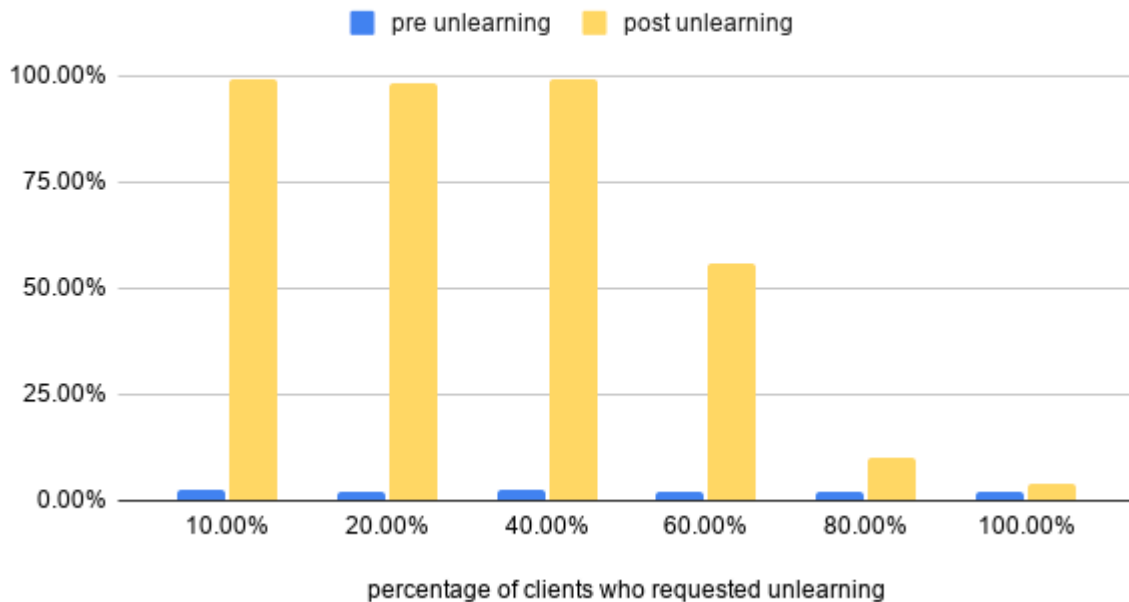
Un *unlearn\_clients\_part* troppo alto corrisponde ad un gran numero di client normali partecipano all'unlearning, diluendo troppo gli effetti dell'attacco.

## ACCURACY



Si nota infatti l'accuracy post unlearning aumentare, poiché più client normali stabilizzano il modello durante l'unlearning.

## ASR



Si nota l'ASR diminuire perché più sono i client normali, più viene contrastato l'attacco.

In questo caso, scegliere la percentuale giusta può aiutare ad evitare il Catastrophic Forgetting.

## Conclusioni :

I dati dimostrano diversi limiti nell'attuazione dell'attacco BADFU in condizioni reali:

- dipende molto dal tipo di learning e unlearning utilizzati:  
da ciò dipende quanto bene il modello impara la backdoor, e quanto bene riesce a eliminare i dati camuffati senza causare una distruzione evidente (Catastrophic Forgetting), attivando così la backdoor.
- emerge che il successo di BADFU dipende molto :
  - dal numero di client bizantini (in questo progetto si nota una soglia del 40%)
  - dalla percentuale di dati avvelenati (in questo progetto del 7.50%)
  - dal numero di epoche di unlearning.  
Tale valore è in una relazione inversamente proporzionale con la percentuale dei client bizantini (all'80% associamo 3 epoche di unlearning, mentre al 40% associamo 7 epoche).
- i client normali possono sia contrastare il successo dell'attacco BADFU diminuendo l'impatto del contributo dei client bizantini, sia garantirne la riuscita.  
Contrastano infatti, in caso di attacchi backdoor troppo aggressivi, il crollo dell'accuracy, permettendone il successo.

- BADFU porta al rischio del Catastrophic Forgetting, ovvero alla distruzione del modello, facilmente.

Si vuole far notare che a percentuali di client bizantini rilevante ( $> 50\% - 80\%$ ) unitamente ad un numero molto alto di epoche ( $> 12$ ) corrisponde un successo massimo dell'attacco, ma un crollo drastico dell'accuracy (Catastrophic Forgetting).

Questo equivale anche a dei falsi positivi nella ricerca: un attacco di BADFU riuscito male può distruggere le prestazioni un modello e rendere la raccolta dei dati difficoltosa.

L'attacco BADFU dimostra la presenza di una falla teorica ed esecutiva nell'architettura del Federated Unlearning, poiché sfrutta proprio l'unlearning per scatenare un comportamento dormiente.

La sua attuabilità rimane molto limitata, poiché l'attaccante dovrebbe controllare una quota sproporzionatamente alta di nodi della rete federata.